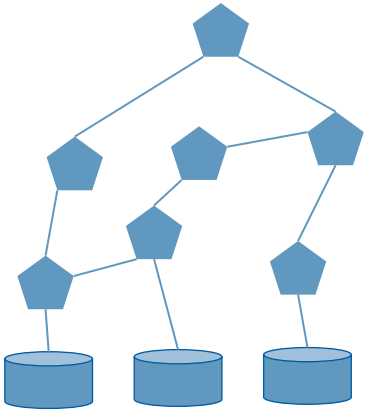
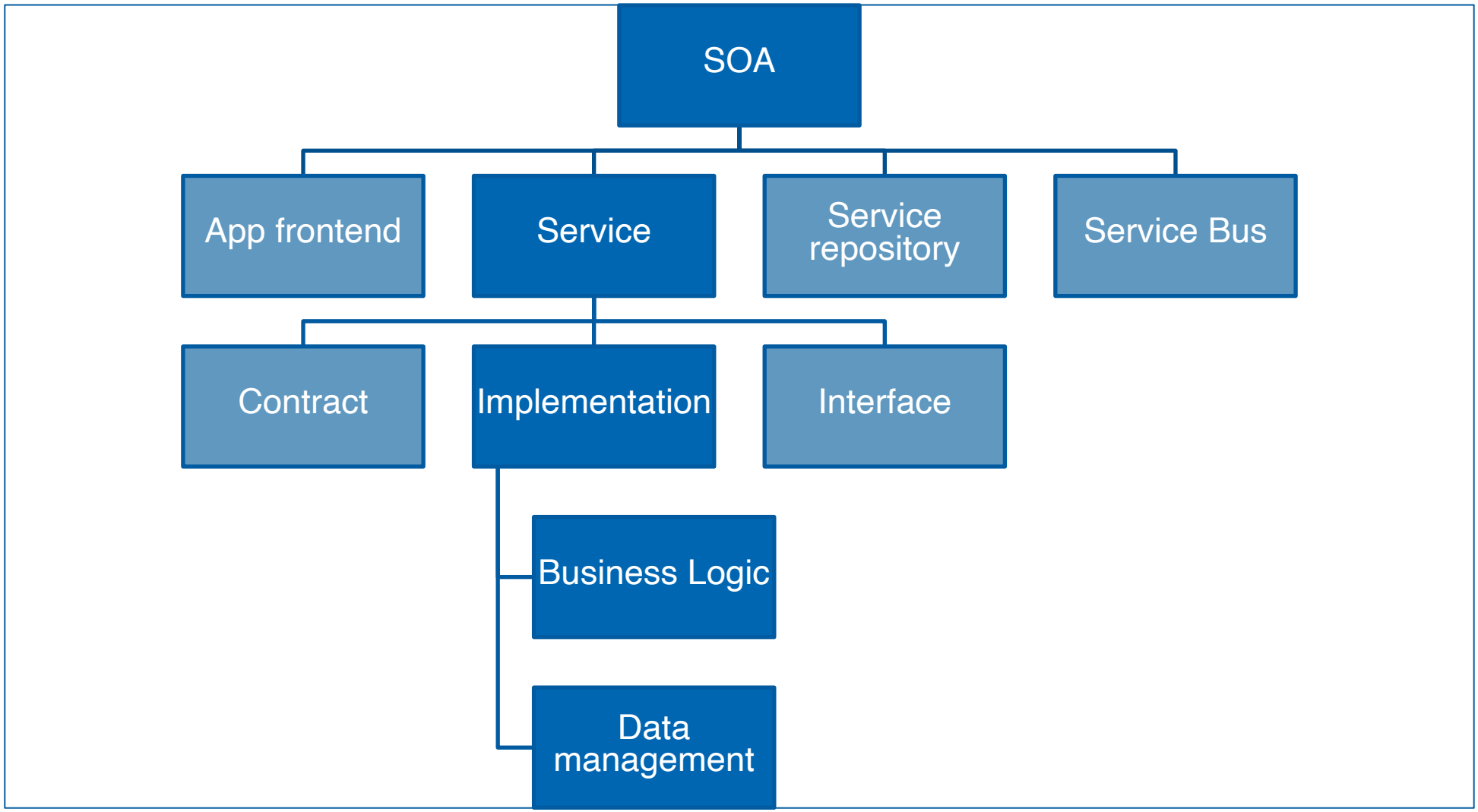
Τεχνολογίες Υπηρεσιών Λογισμικού

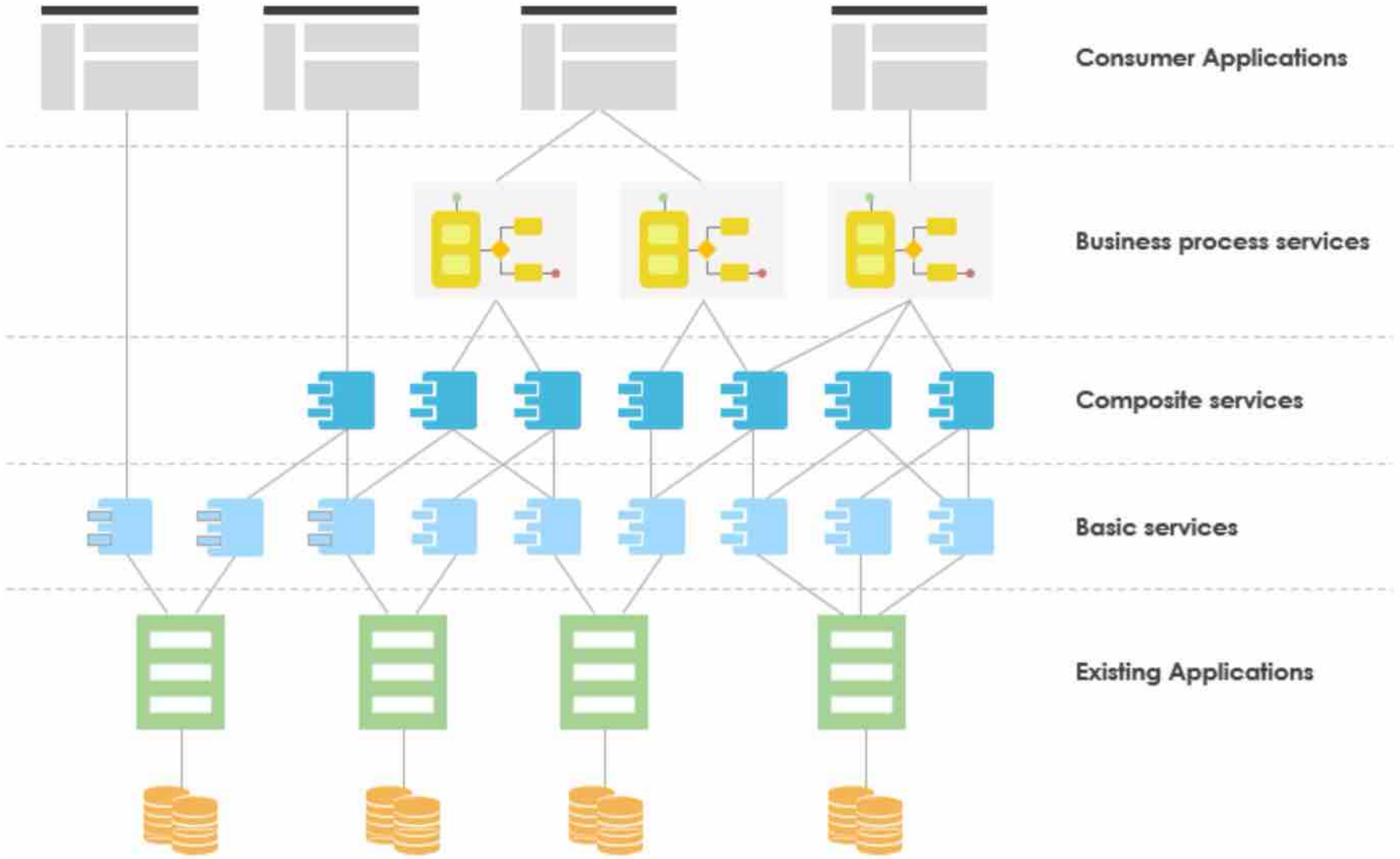


Microservices II

Service-Oriented Architecure



SOA: Service-Oriented Architecture



Source: visual-paradigm.com

Microservices

SOA at a different scale

- Same design principles except that one microservice may not offer complete business function(s)
- Small size (goes with the name)
- Reusability is not a must (possible code duplication)
- Asynchronous communication (no service bus) enables better fault tollerance
- Data is available *inside* each microservice: possible data duplication, complexity

Microservice architecture

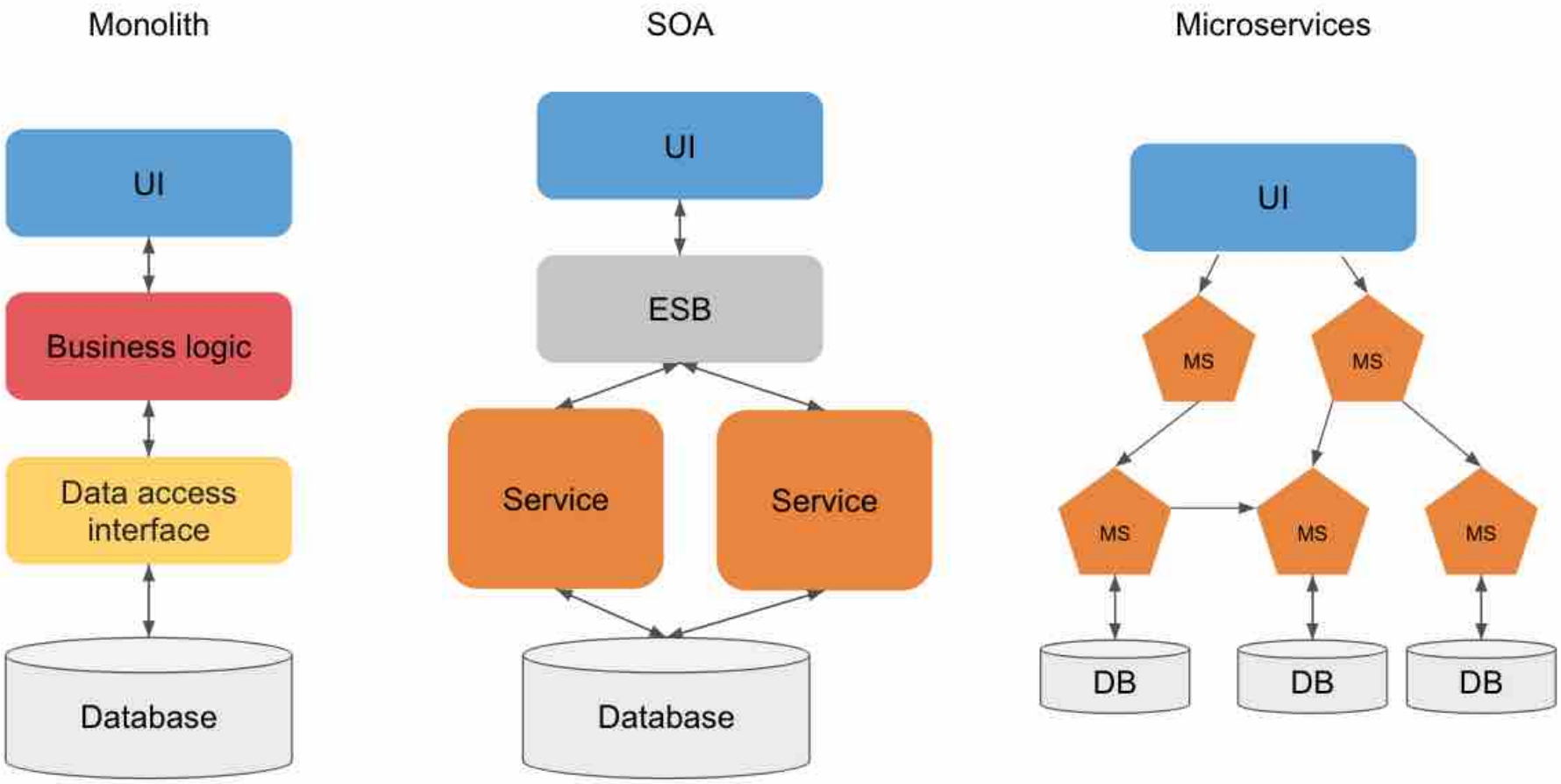
Focus on application architecture

- Break down into small, isolated functions and expose as “services”
- Develop & deploy independently!

Each service has

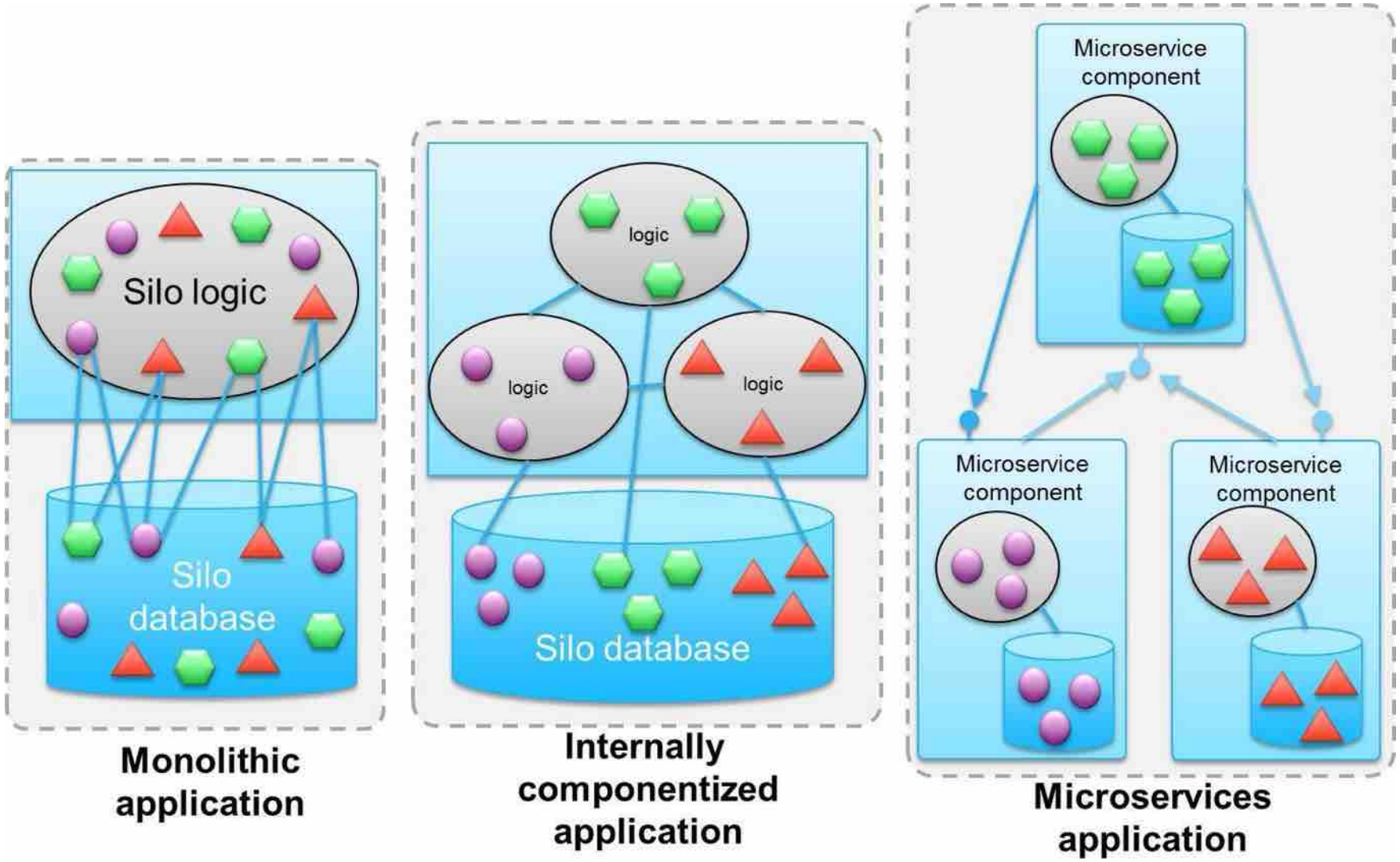
- A single responsibility
- Access to the data it needs
- Limited business functionality
- Limited re-usability
- Maximized agility & resilience

Monoliths, SOA & microservices illustrated



Source: Microservices Architecture From A to Z, S.Bouttier @Medium

Monoliths, SOA & microservices illustrated



Source: IBM

Microservices

Pros

- Independend development, deployment, scaling
- Small teams, no sharing of source code, simpler code, data isolation (many databases)
- Resilience by reducing impact when a service fails
- Easier to implement in cloud ecosystems

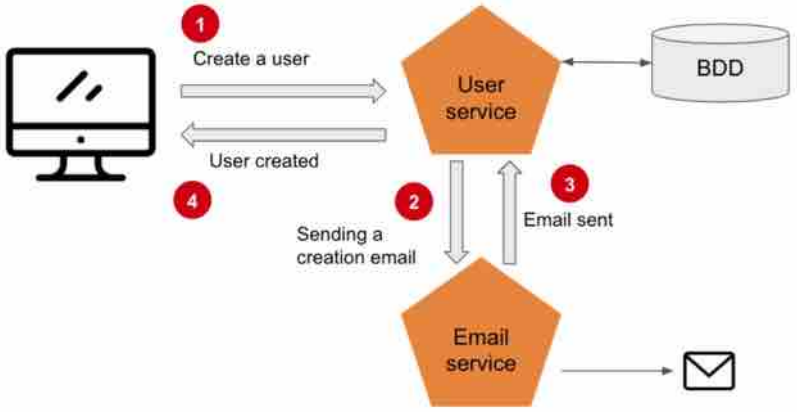
Cons

- System complexity, deployment management
- Testing becomes more complicated
- Distributed databases have data integrity challenges, especially for transaction-based applications
- Network latency, critical for complex communications between services

No universal rule, choice depends on the context!

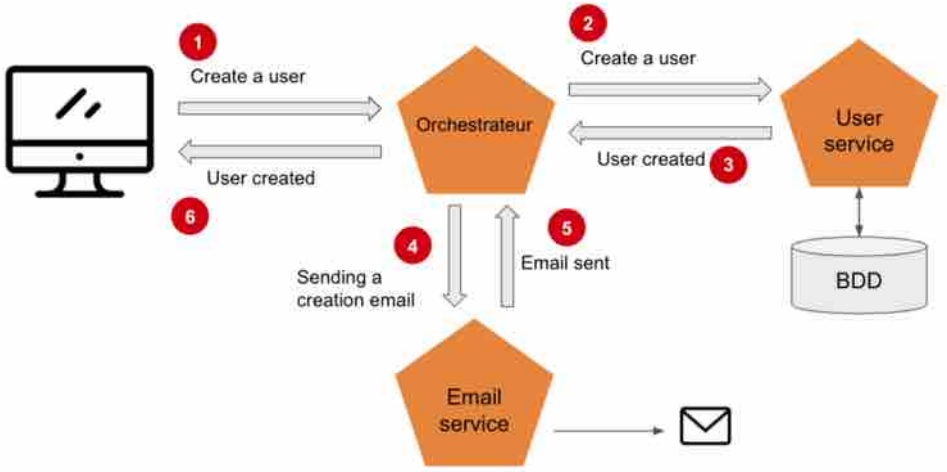
Microservices communication

Use of REST calls vs...



- Dependencies become complex
- Complex is difficult to maintain
- Behaviour is impacted
- Can do better

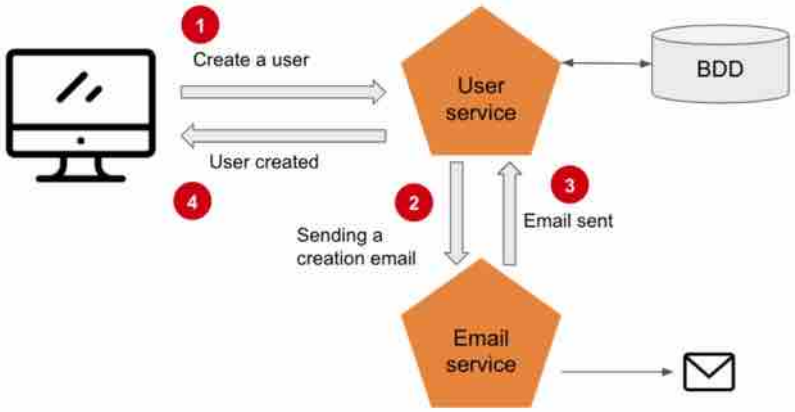
1: orchestration



- Add a service to manage dependencies ("orchestrator")
- The orchestrator should not fail

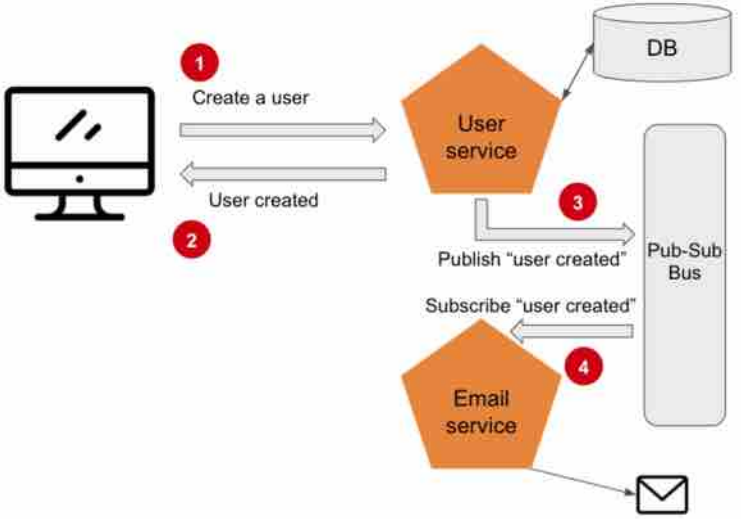
Microservices communication

Use of REST calls vs...



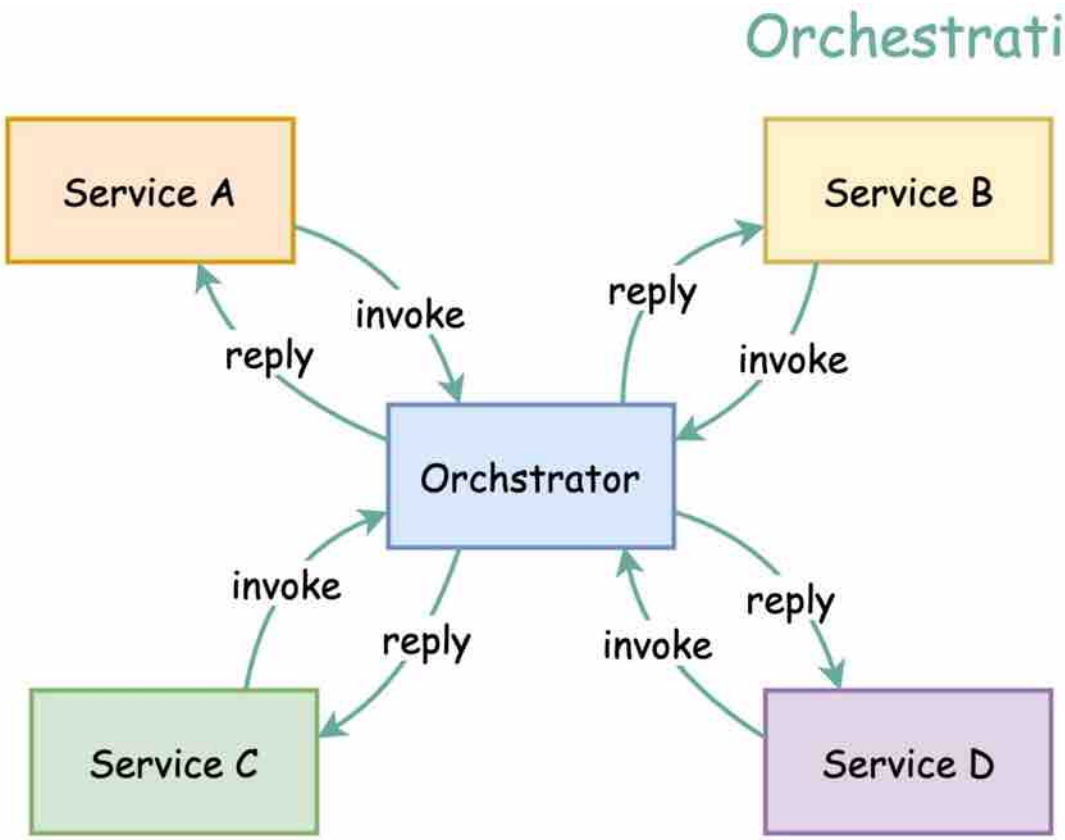
- Dependencies become complex
- Complex is difficult to maintain
- Behaviour is impacted
- Can do better

2: choreography



- Use a publish-subscribe model (no direct or mediated communication)
- Data consistency concerns

Orchestration

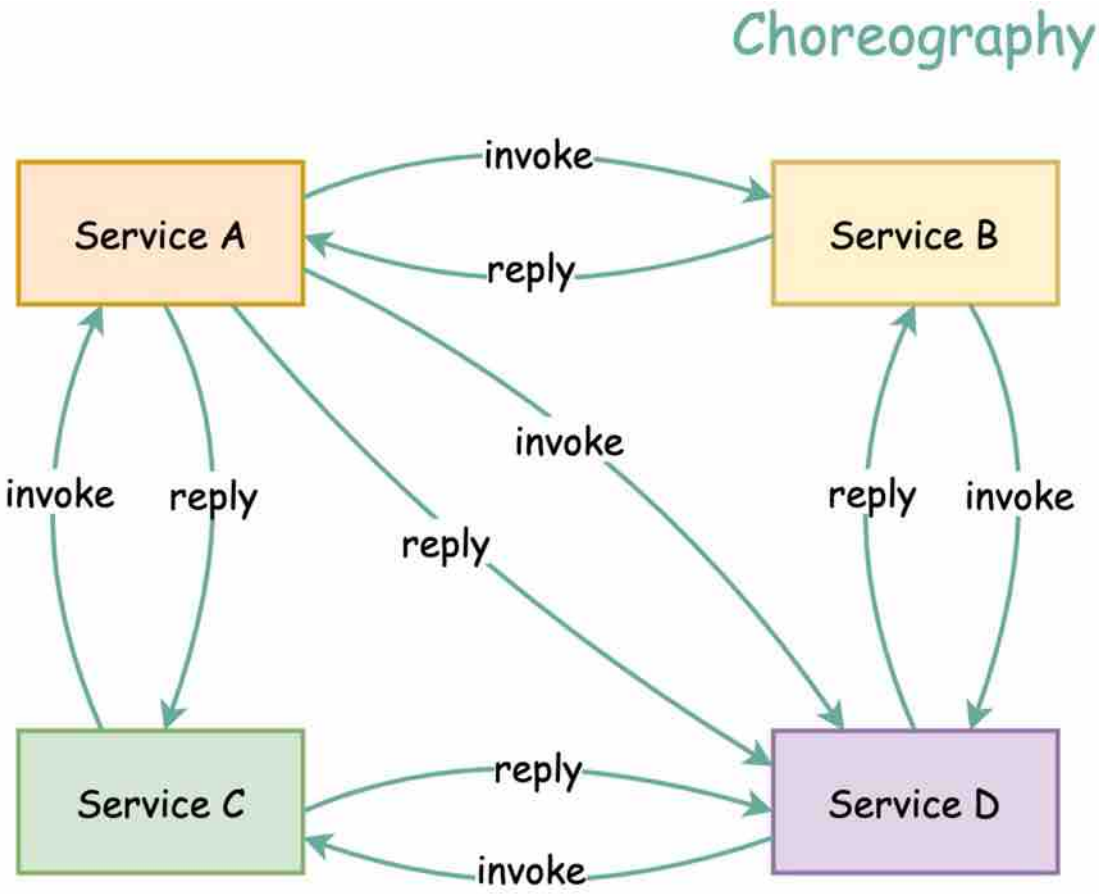


1. services more loosely coupled

2. domain boundaries are defined clearly, and test cases can be confined in the domains

Images: bytebytego.com

Choreography, the hard way (to avoid!)

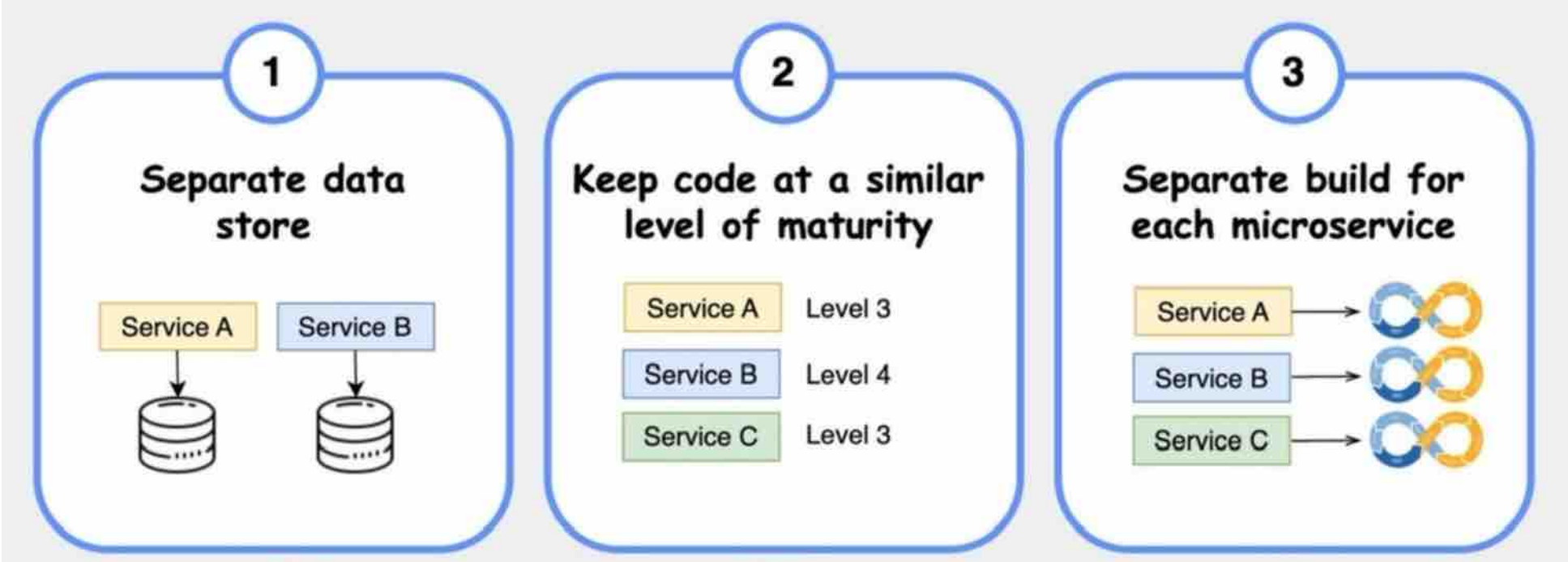


1. service dependency is complicated which makes debugging and testing more difficult

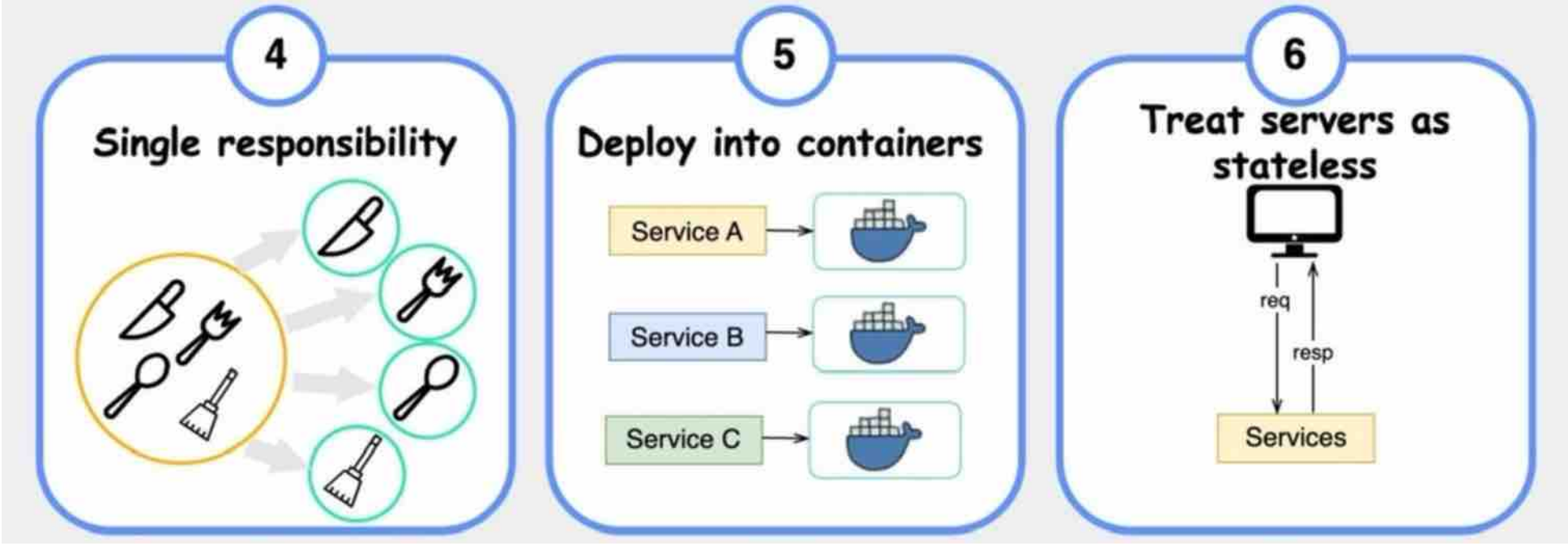
2. point-to-point communications

3. fault tolerance scenarios are complex

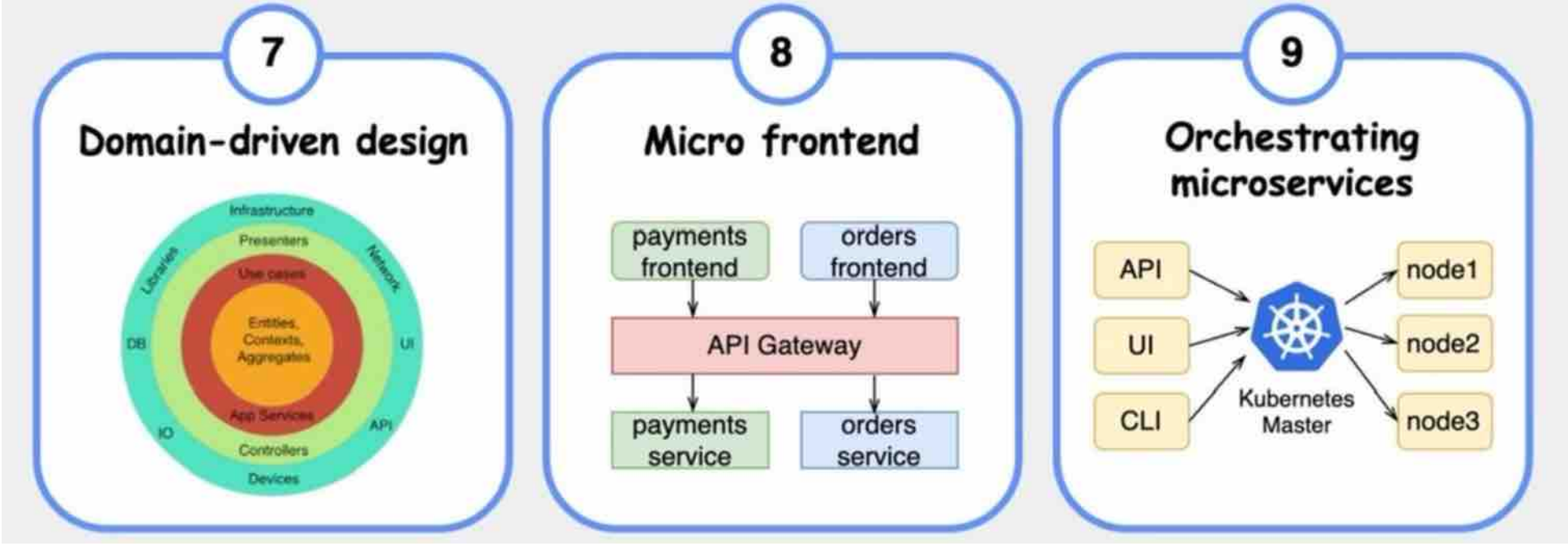
Microservices "best practices"



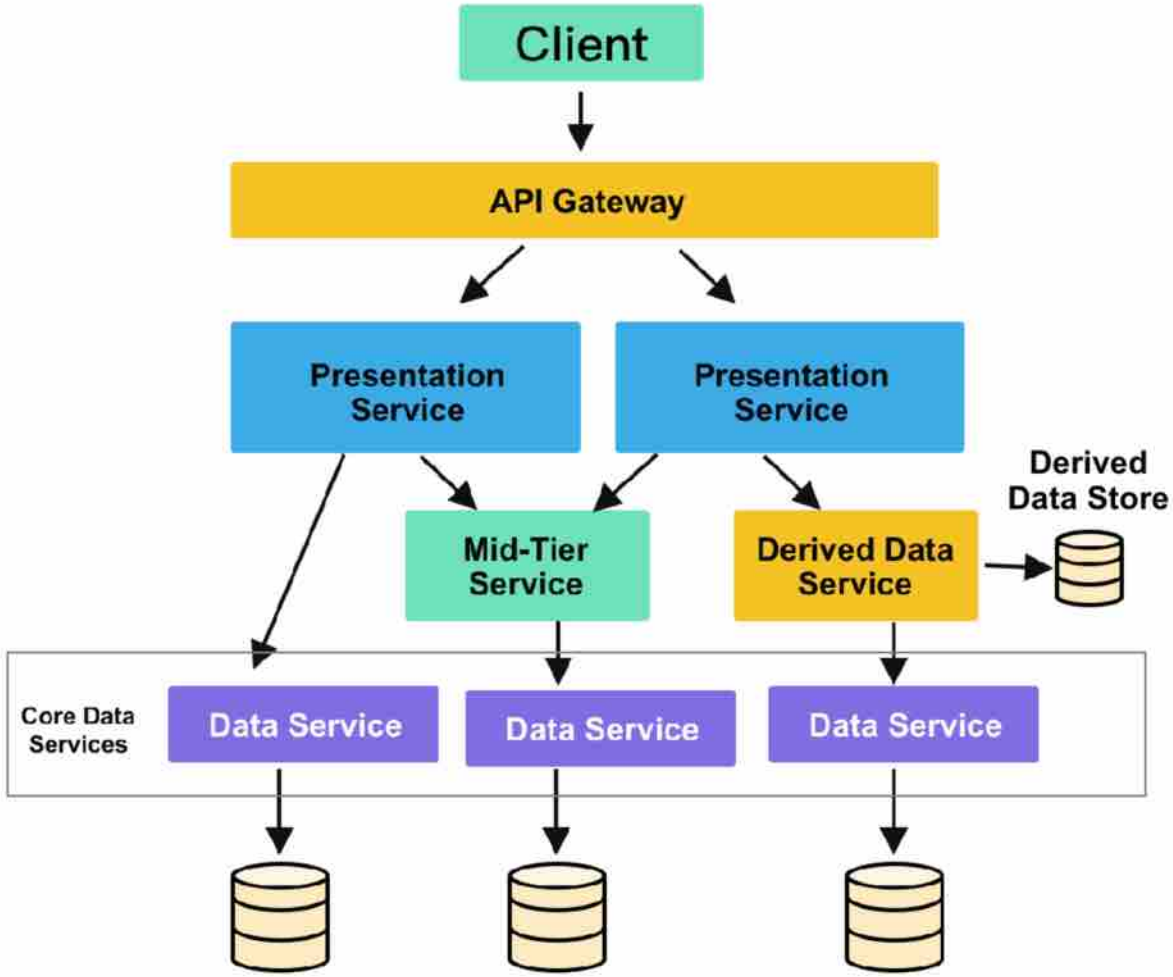
Microservices "best practices"



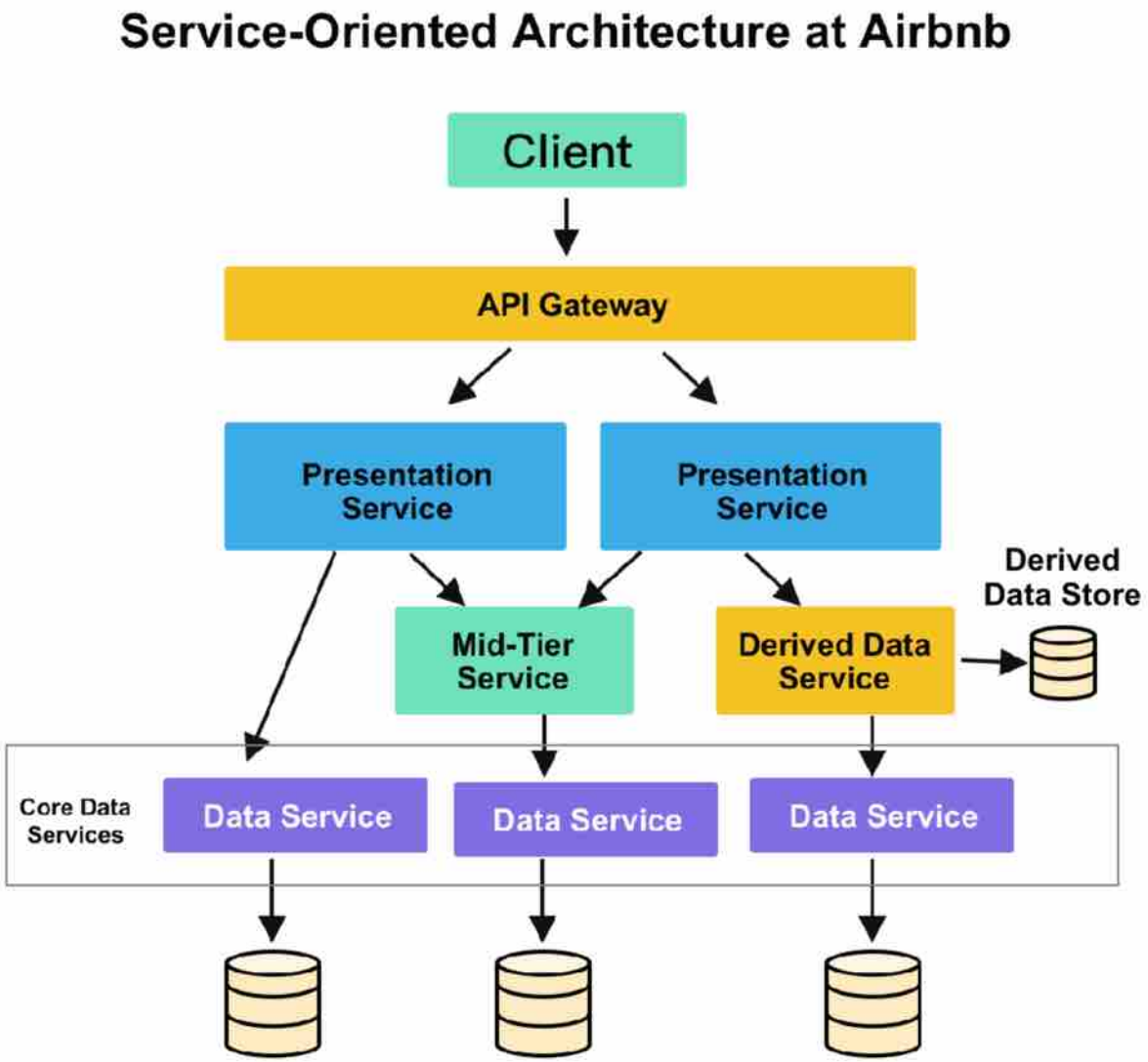
Microservices "best practices"



Is this SOA or Microservices?



Is this SOA or Microservices?



Data Service - This is the bottom layer and acts as the entry-point for all read and write operations on the data entities. A data service must not be dependent on any other service because it only accesses the data storage.

Derived Data Service - The derived services stay one layer above the data service. These services can read from the data services and also apply some basic business logic.

Middle Tier - They are used to house large pieces of business logic that doesn't fit at the data service level or the derived data service level.

Presentation Service - At the very top of the structure are the presentation services. Their job is to aggregate data from all the other services. In addition, the presentation service also applies some frontend specific business logic before returning the data to the client.

SOA/Microservices example

Q2D: Quizzes, Quick and Dirty ☺

- A quiz contains only multiple-choice questions
- Only 4 options per question allowed
- Each option has its own grade
- Questions cannot be re-used in other quizzes
- Users can answer each question only once

Q2D revisited

Discover logic

- login
- register
- answer
- create quizzes
- create questions
- display grades

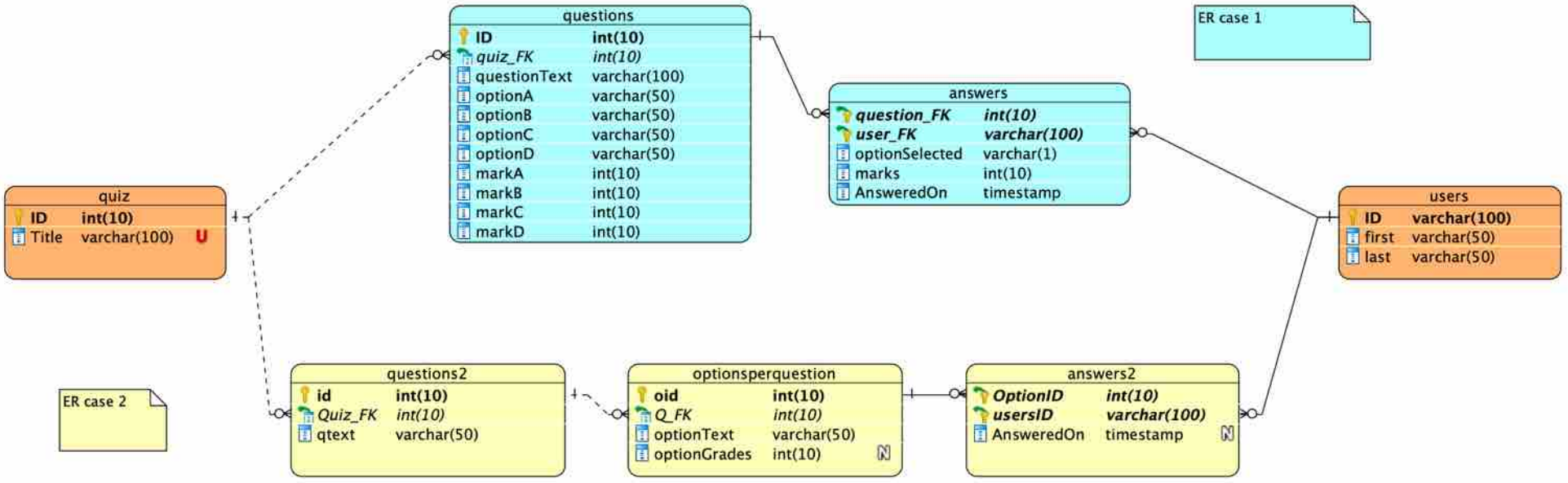
Q2D revisited

SOA principles:

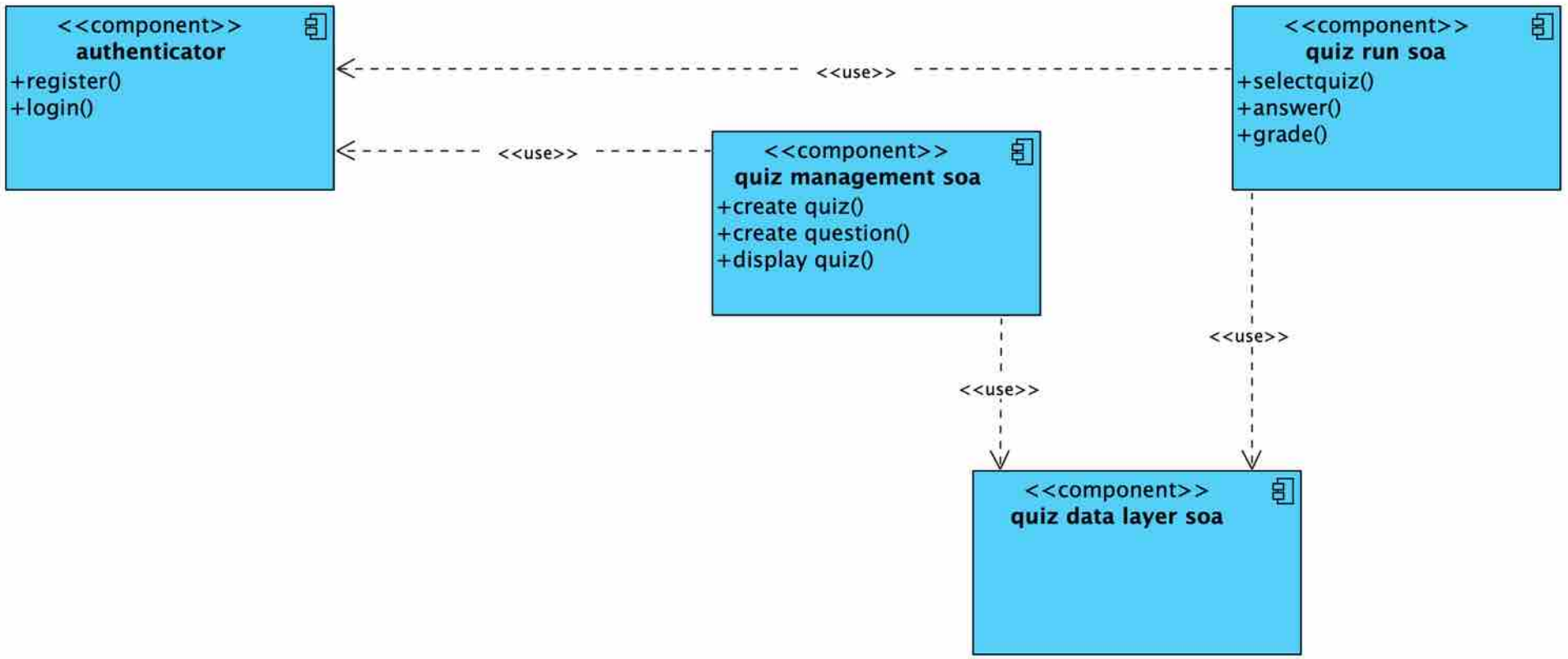
Discover possibly re-usable services

- login
- register
- create questions
- answer questions
- grading
- analytics

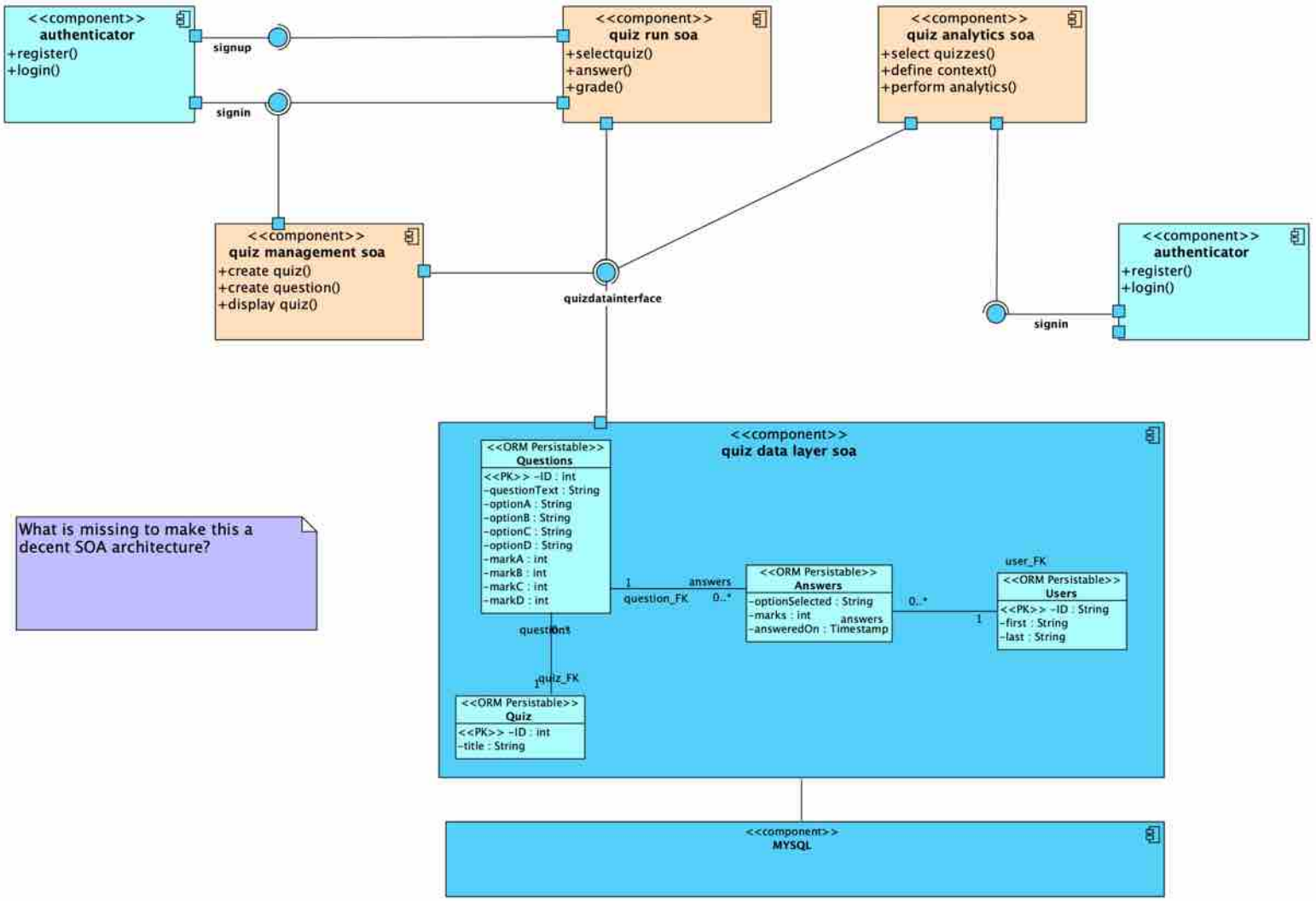
Q2D revisited: understanding data



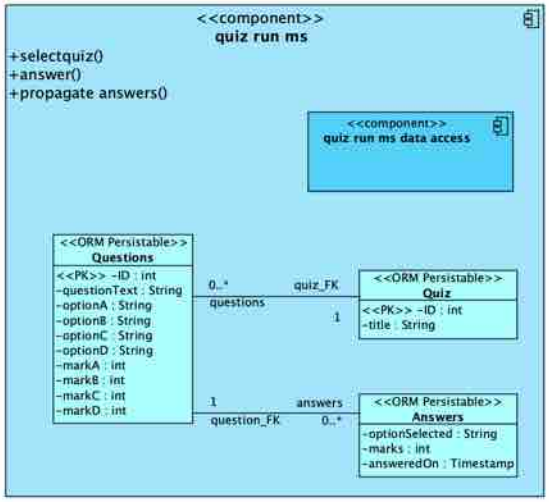
Q2D revisited: SOA

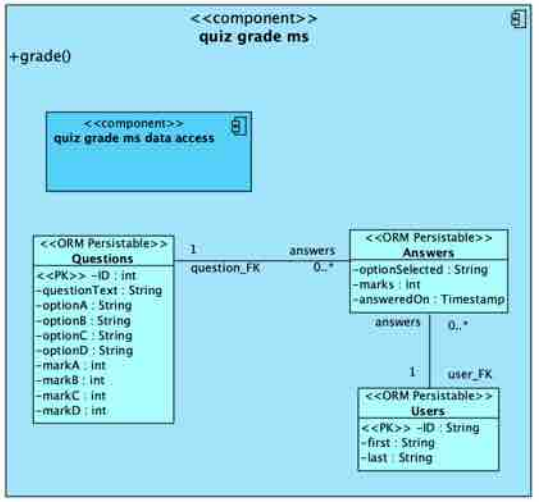


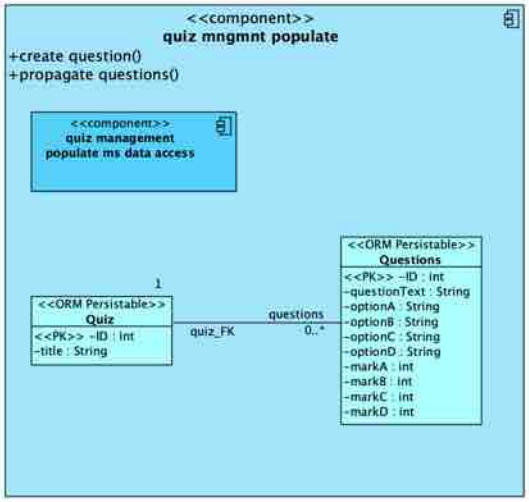
Q2D revisited: SOA II

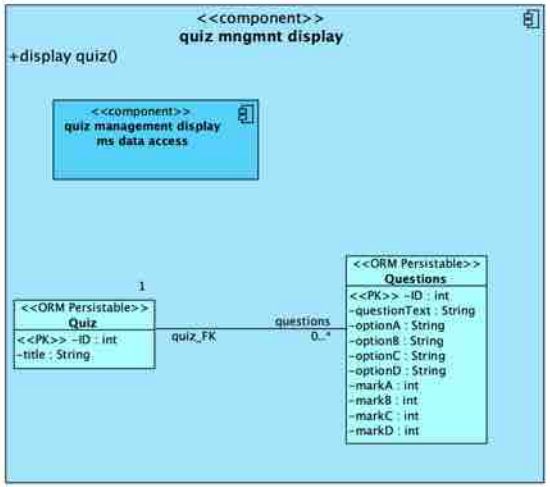


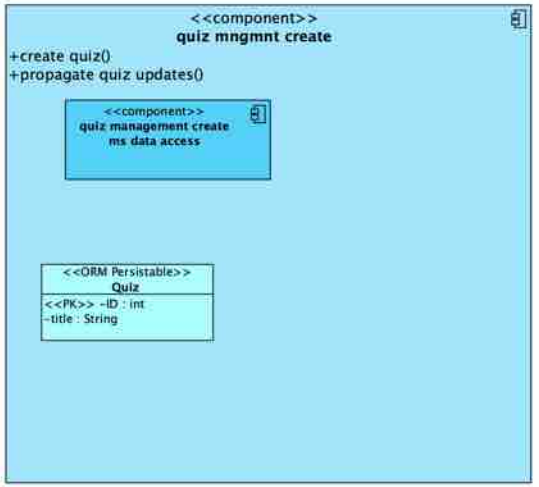
Q2D: microservices definition (wip)

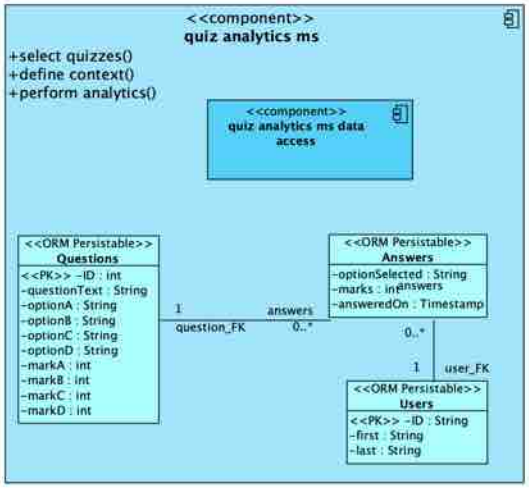




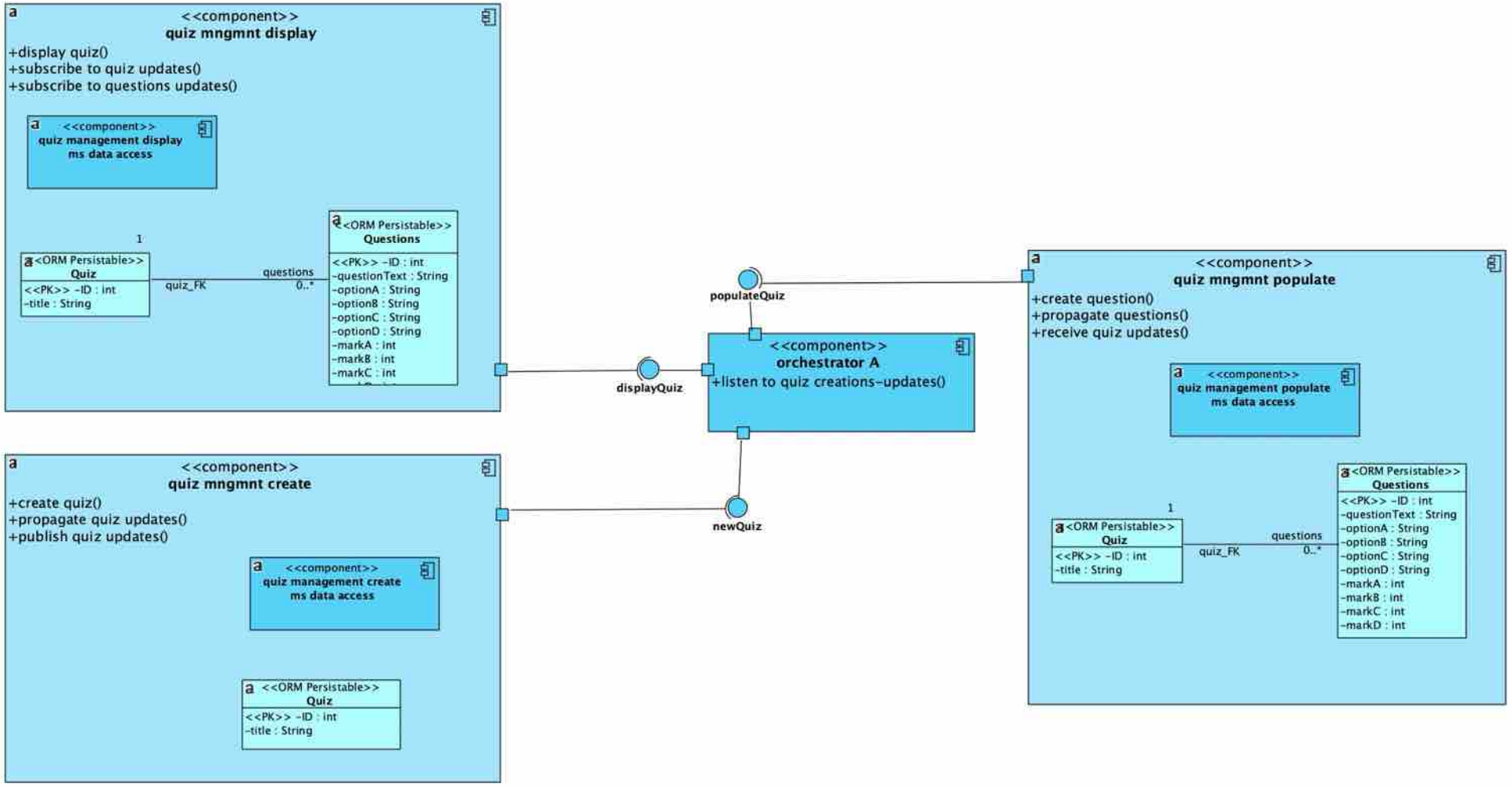




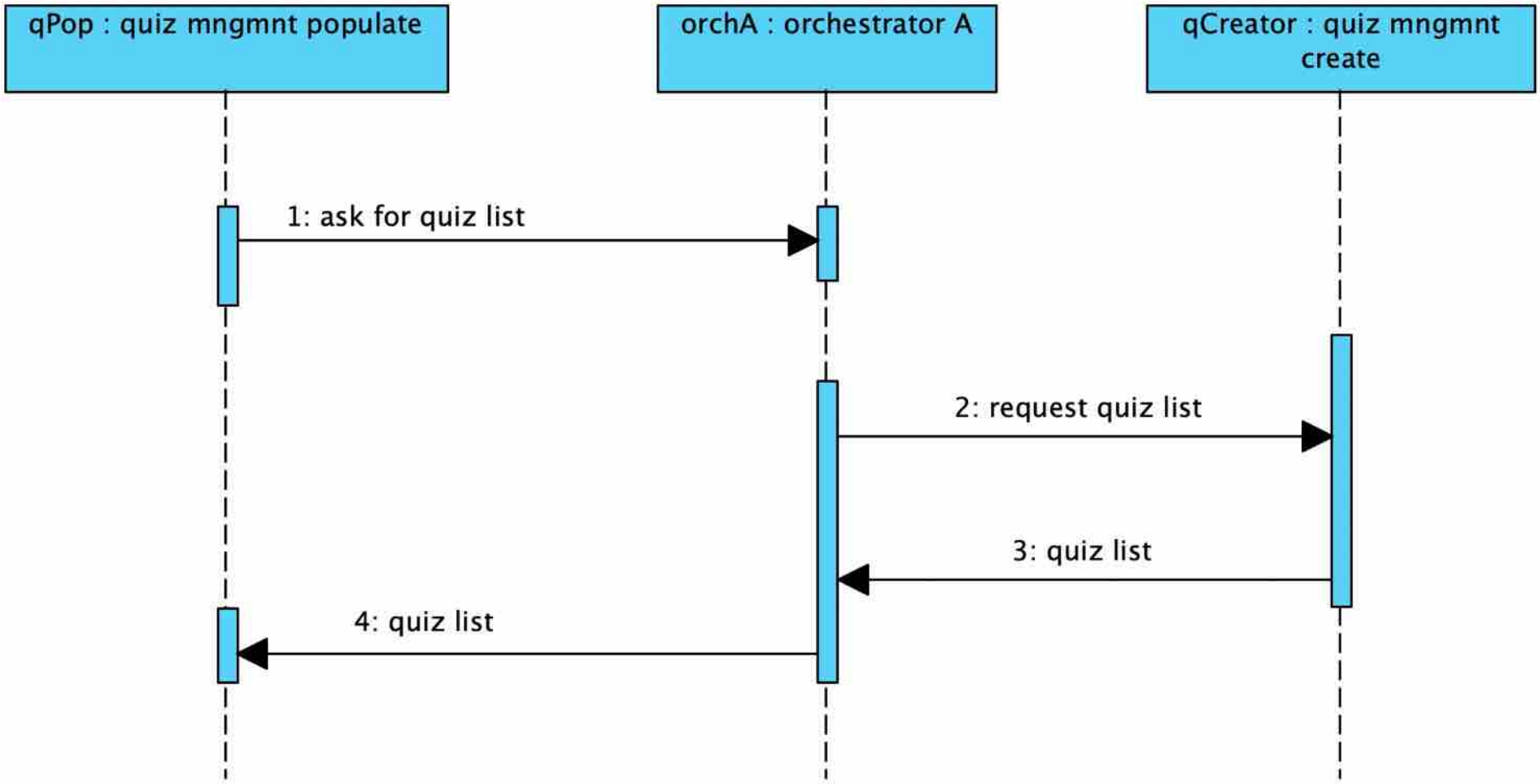




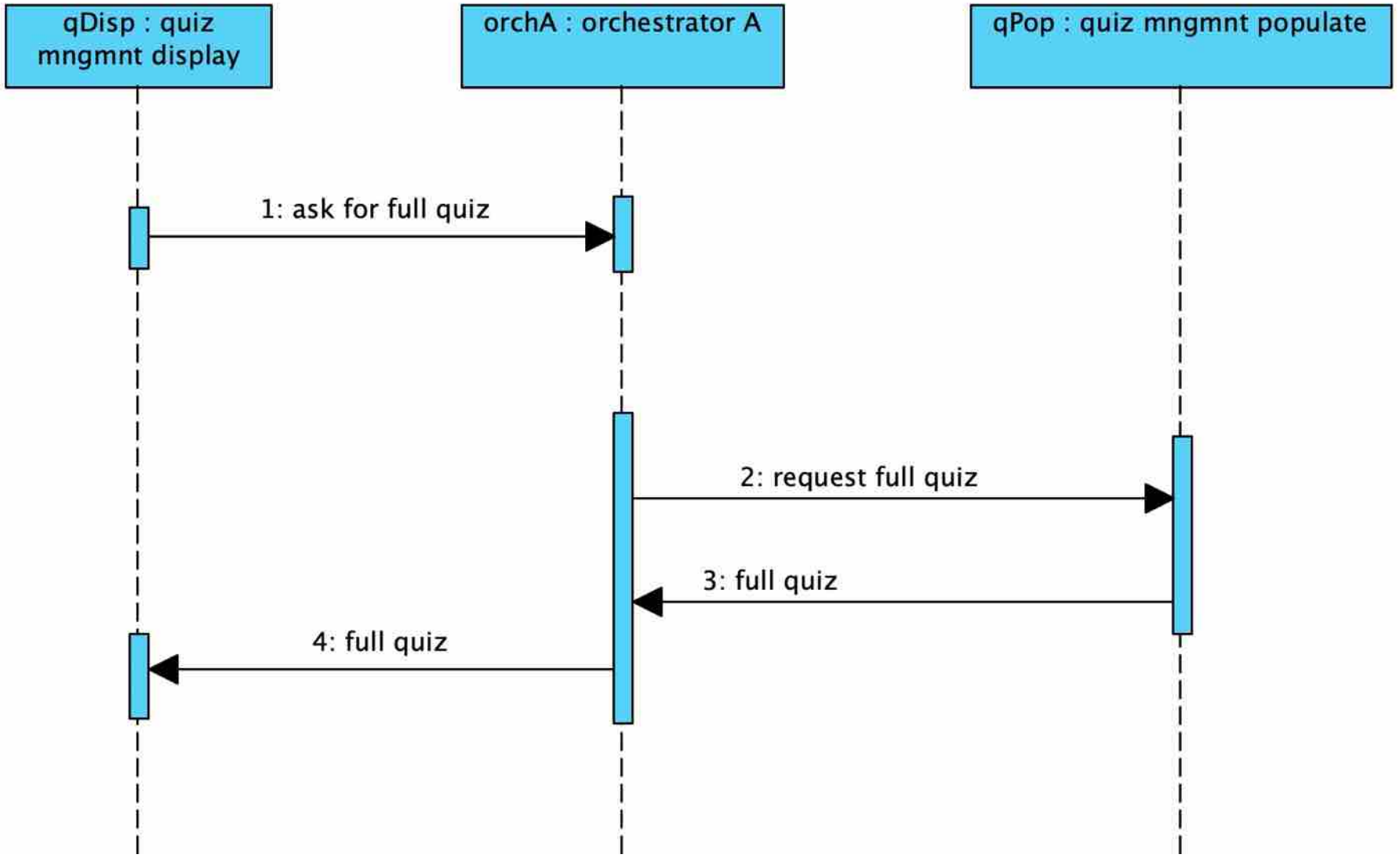
Q2D architecture – orchestrator (partial)



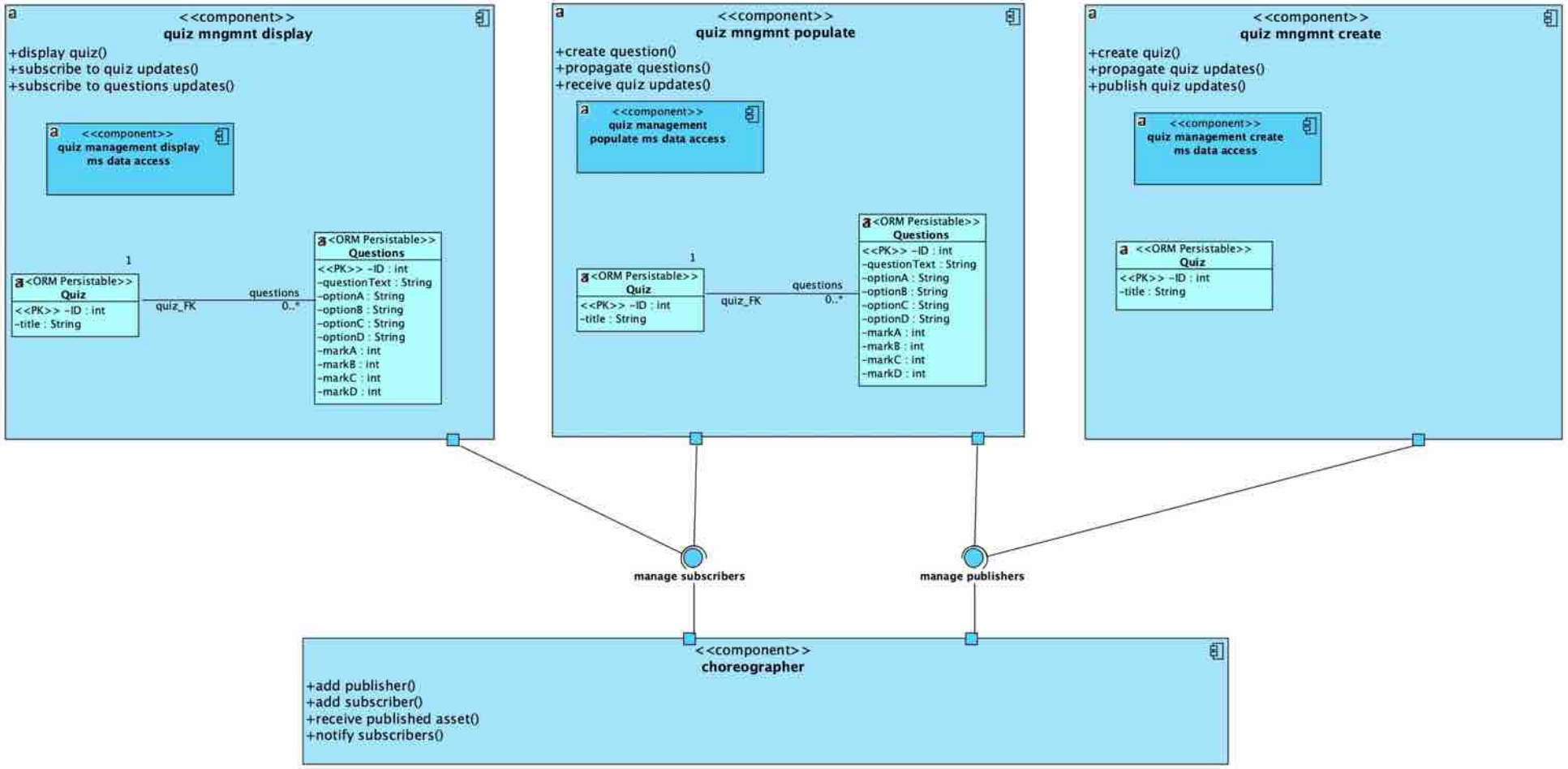
Q2D sequence – orchestrator (partial)



Q2D sequence – orchestrator (partial)



Q2D architecture – choreographer (partial)



Q2D is available on…

- github.com/ntua/saas-q2d
  UML diagrams
- hub.softlab.ntua.gr/hub/projects/saas_Q2D_example
  Agile project tracking

Are LLMs any useful for Microservices?

Ongoing investigation based on a past SaaS project

Microservices architectures from LLMs

Investigation in progress

- FR/NFR texts or SRS
- RAG materials & methods
- LLM proposes MS architecture as PlantUML
- Qualitative assessment & metrics

Topic

- Chart generation online service: MyCharts
- SaaS course semester project 2022-2023

MyCharts (2022-2023 project)

The project focuses on the development of "myCharts", a small web-based software application to be provided as SaaS, whose specifications are provided by the instructors. The development cycle does not include requirements analysis, but aims to simulate conditions where, during the development of an application, each team specifies the specifications that are sufficiently known at the beginning. The focus is on the architecture, technologies, and application development tools provided as services, as well as the application of flexible (agile) development methodologies, also with the corresponding modern tools.

"MyCharts" is a web-based application/service that is aimed at users lacking the requisite technical expertise, enabling them to generate charts and download them in various formats onto their computer systems. The service supports a range of 3 to 9 chart types, depending on the number of students within each group. For each type, the user has the option to download a template file in csv format, illustrating how to populate chart data as well as providing supported formatting guidelines. To create a chart, the user must upload the completed csv file. Upon successful generation of the chart, the user can proceed to download the chart in one of the supported formats.

The minimum supported features are listed below:

- User management and login with a Google account
- Downloading a template file of data and formatting parameters for each supported chart type
- Uploading data and formatting parameter files
- Creating a chart and saving it to the server in supported formats such as pdf, png, svg, and html (for adding to a website)
- Displaying the charts created by each user and allowing them to download the selected format
- Managing quotas that allow the creation of charts.

The aforementioned functionalities are offered via three distinct use cases, which are adequately documented in the Software Requirements Specification (SRS) document provided within the Helios software suite.

MyCharts: FR, NFR

FUNCTIONAL REQs

- Authenticate users via Google accounts.
- Allow download of CSV templates for 3 supported chart types (Basic line, Line with annotations, Basic column).
- Upload CSV file with the user data for chart generation.
- Validate uploaded CSV files against the template structure, data types, and mandatory data existence.
- Generate charts with data from the CSV file uploaded by the user, using Highcharts.
- Save charts to the server in PDF, PNG, SVG, and HTML formats.
- Display a dashboard with the history of user-generated charts, including chart previews.
- Download selected chart type.
- Charge quotas for chart creation.
- Allow users to delete or download charts.
- Sell quotas and receive payment by a payment gateway
- Maintain user profiles containing the following data: Name (from Google account), profile picture (from Google account), email (from Google account), remaining quota, account creation timestamp, last login timestamp.
- Display user's profile info, including remaining quotas.

NON-FUNCTIONAL REQs

- Performance: Chart generation completes within 3 seconds.
- Usability: Intuitive UI with guided workflows (e.g., tooltips, validation hints) to assist non-technical users.
- Security: SSL encryption: Data transmitted ove the internet is encrypted using SSL.
- Availability: The service should be available 90% of the time on a daily basis.
  - Availability over Consistency during network partitioning.
  - While in network partitioning, although temporary data inconsistencies may occur, as many as possible services should remain fully operational.
- Scalability: 1000 concurrent users

MyCharts: SRS

1. Introduction

1.1 Purpose

The MyCharts Application is a web-based service designed to enable users with minimal technical expertise to generate, manage, and download charts in various formats. It simplifies chart creation by providing templates for source data, supporting data uploads. It uses the Highcharts library for chart generation.

1.2 Scope

This application allows users to:

- Download CSV templates for supported chart types.
- Upload CSV files to generate charts.
- Save and download charts in PDF, PNG, SVG, and HTML formats.
- Purchase quotas for chart creation.
- View and download generated charts.

1.3 Definitions, Acronyms, and Abbreviations

- **CSV**: Comma-Separated Values file format.
- **Quota**: A limit on the number of charts a user can create.
- **Highcharts**: JavaScript library used for chart rendering.

1.4 References

- Highcharts Library: https://www.highcharts.com
- Google Authentication API: https://developers.google.com/identity
- CSV Format Specification: https://tools.ietf.org/html/rfc4180

1.5 Overview

This document outlines functional and non-functional requirements, use cases, and interface specifications for the MyCharts Application.

MyCharts: SRS

2. System Overview

2.1 Product Perspective

MyCharts is a SaaS (Software as a Service) standalone web application. It will be offered online and produce revenue by selling credits for chart generation.

2.2 Product Functions

1. User authentication via Google accounts.
2. Download CSV templates for chart types.
3. Upload CSV files to generate charts.
4. Save charts in PDF, PNG, SVG, and HTML formats.
5. Display and manage user-generated charts.
6. Charge for quotas needed to create charts.

2.3 User Characteristics

Primary users include educators, students, and professionals needing simple chart generation. No coding or technical skills are required.

2.4 Operating Environment

- **Frontend**: Modern browsers (Chrome, Firefox, Safari).
- **Backend**: Application Server with Node.js or Python runtime and database server(s) as needed.
- **Network**: Internet connectivity required for service offering.

2.5 Assumptions and Dependencies

- Google OAuth is available for user authentication.
- Highcharts library is accessible for chart rendering.
- Users follow the CSV structure.

MyCharts: SRS

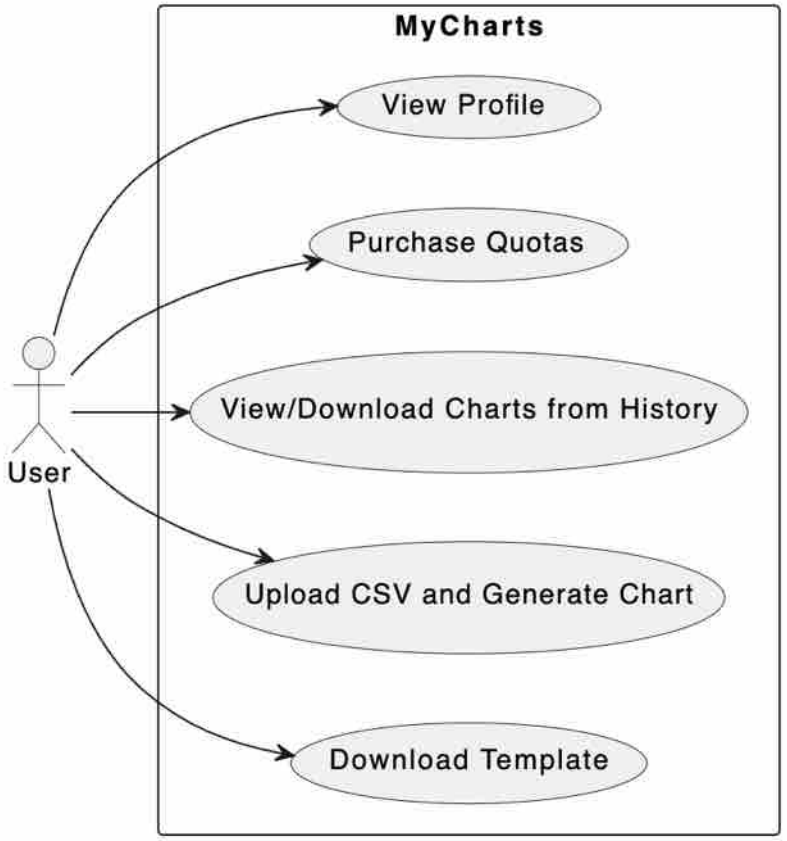
3.1 Functional Requirements Specification

1. Authenticate users via Google accounts.
2. Allow download of CSV templates for 3 supported chart types (Basic line, Line with annotations, Basic column).
3. Upload CSV file with the user data for chart generation.
4. Validate uploaded CSV files against the template structure, data types, and mandatory data existence.
5. Generate charts with data from the CSV file uploaded by the user, using Highcharts.
6. Save charts to the server in PDF, PNG, SVG, and HTML formats.
7. Display a dashboard with the history of user-generated charts, including chart previews.
8. Download selected chart type.
9. Charge quotas for chart creation.
10. Allow users to delete or download charts.
11. Sell quotas and receive payment by a payment gateway
12. Maintain user profiles containing the following data: Name (from Google account), profile picture (from Google account), email (from Google account), remaining quota, account creation timestamp, last login timestamp.
13. Display user's profile info, including remaining quotas.

MyCharts: SRS

3.2 Use Cases

3.2.1 Use Case Diagram



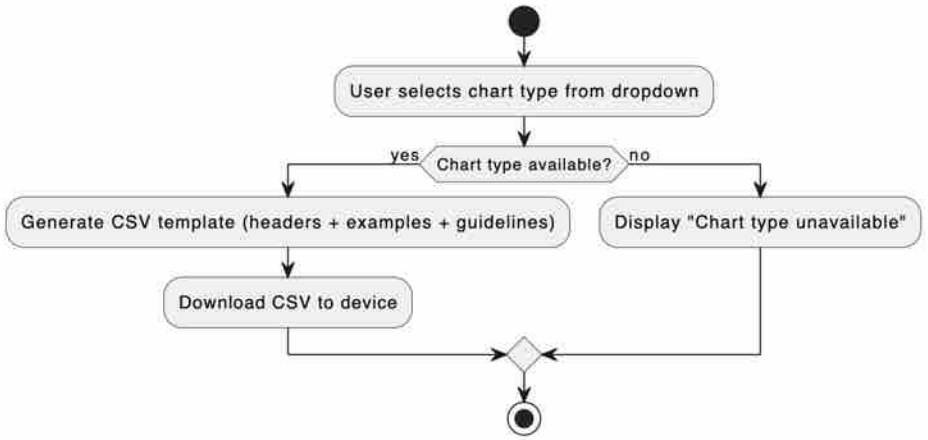

```
### 3.2 Use Cases

#### 3.2.1 Use Case Diagram
```plantuml
@startuml
left to right direction
actor "User" as user
rectangle MyCharts {
  usecase "Download Template" as Download
  usecase "Upload CSV and Generate Chart" as Upload
  usecase "View/Download Charts from History" as History
  usecase "Purchase Quotas" as Quota
  usecase "View Profile" as Profile
}
user --> Download
user --> Upload
user --> History
user --> Quota
user --> Profile
@enduml
```
```

MyCharts: SRS

3.2.2 Use Case 1: Download CSV Template

- **Actors**: User
- **Preconditions**:
  a) User is logged in.
- **Main Flow**:
  1. User navigates to the "Create Chart" section and selects a chart type from a dropdown menu.
  2. System displays the 3 supported chart types.
  3. User clicks "Download Template" for the selected chart type.
  4. System provides a CSV file for each chart type, containing:
     - Predefined column headers matching the chart type.
     - Sample data rows, to be replaced by the user's actual data.
     - Chart formatting parameters.
  5. CSV file is downloaded to the user's device.
- **Post-conditions**:
  a) CSV template is saved locally on the end-user's machine for data entry.
- **Activity Diagram**:



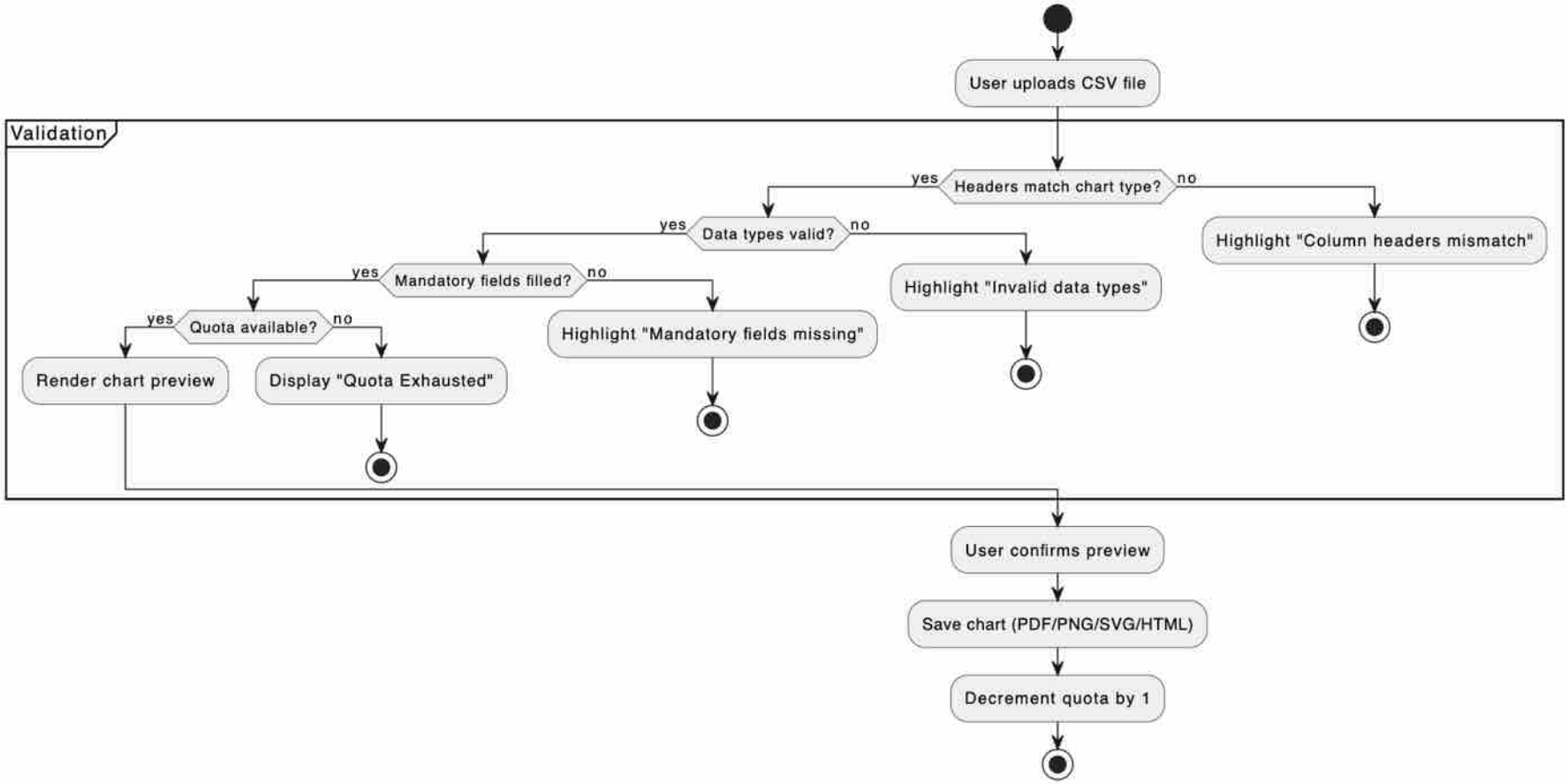
MyCharts: SRS

3.2.3 Use Case 2: Upload CSV and Generate Chart

- **Actors**: User
- **Preconditions**:
  a) User has a valid CSV file formatted per the template.
  b) User has remaining quota for chart creation.
- **Main Flow**:
  1. User navigates to the "Generate Chart" section and uploads a CSV file.
  2. System performs validation checks:
     - Column headers match the selected chart type.
     - Data types are correct (e.g., numeric values for data).
     - No empty mandatory fields.
  3. System renders a preview of the chart using Highcharts.
  4. User confirms the preview.
  5. System saves the chart to the server in all supported formats (PDF, PNG, SVG, HTML).
  6. User's quota is decremented by 1.
- **Alternate Flows**:
  - **Invalid CSV**: System highlights errors (e.g., "Column 'Revenue' missing") and blocks submission.
  - **Quota Exhausted**: System displays "Quota Limit Reached: Upgrade to create more charts."
- **Post-conditions**:
  a) Chart is stored on the server.
  b) Chart appears in the user's dashboard.

MyCharts: SRS

- **Activity Diagram**:



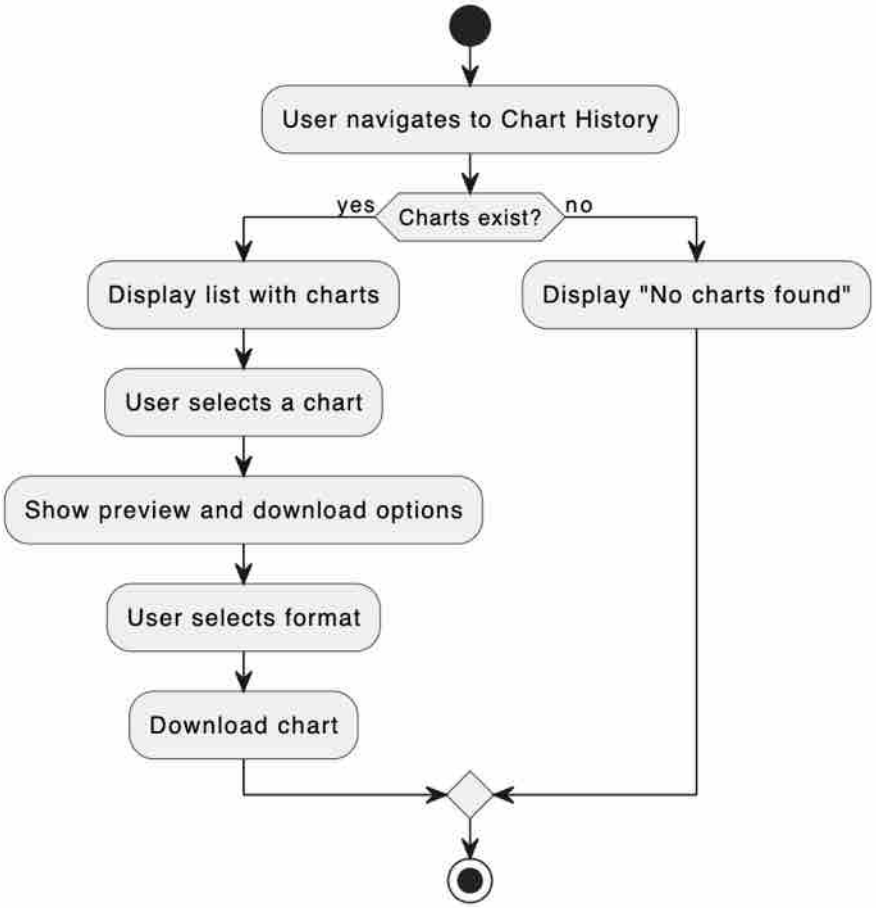
MyCharts: SRS

3.2.4 Use Case 3: View/Download Chart from History

- **Actors**: User
- **Preconditions**:
  a) User is logged in.
  b) At least one chart has been previously generated by the logged in user.
- **Main Flow**:
  1. User navigates to the "Chart History" section in the dashboard.
  2. System displays a list of all previously generated charts with timestamps.
  3. User selects a chart from the list.
  4. System displays a preview and provides download options (PDF, PNG, SVG, HTML).
  5. User selects a format to download the chart.
- **Alternate Flow**:
  - If no charts exist, the system displays a message: "No charts found in history."
- **Post-conditions**:
  a) The selected chart is downloaded in the chosen format.

MyCharts: SRS

- **Activity Diagram**:



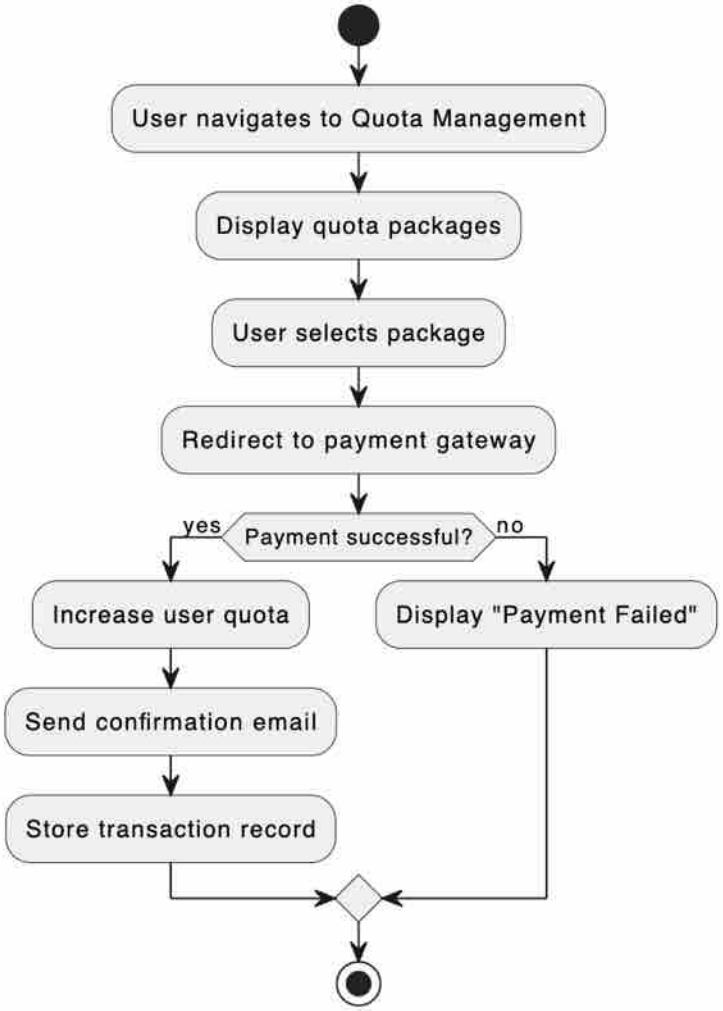
MyCharts: SRS

3.2.5 Use Case 4: Purchase Additional Quotas

- **Actors**: User
- **Preconditions**:
  a) User is logged in.
  b) Payment gateway integration is operational.
- **Main Flow**:
  1. User navigates to the "Quota Management" section in the dashboard.
  2. User selects the number of quotas to purchase and confirms the purchase.
  3. System redirects the user to a secure payment gateway (e.g., Stripe/PayPal).
  4. User completes the payment process.
  5. System validates the payment and increases the user's quota limit accordingly.
  6. System sends a confirmation email with a receipt.
- **Alternate Flows**:
  - **Payment Failure**: If payment fails, the system retains the original quota and notifies the user.
  - **Partial Payment**: System cancels the transaction and restores the initial quota state.
- **Post-conditions**:
  a) The user's quota limit is updated.
  b) A transaction record is stored in the system.

MyCharts: SRS

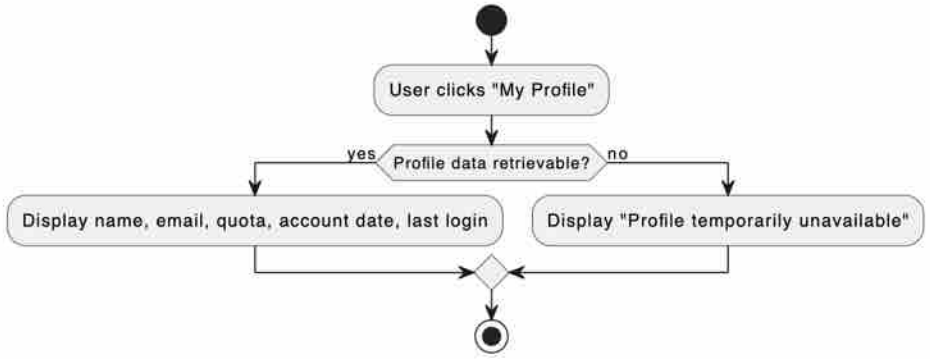
- **Activity Diagram**:



MyCharts: SRS

3.2.6 Use Case 5: View My Profile

- **Actors**: User
- **Preconditions**:
  a) User is logged in.
- **Main Flow**:
  1. User clicks the "My Profile" button in the dashboard navigation menu.
  2. System retrieves and displays the following profile details:
     - Name and profile picture (from Google account).
     - Email address associated with the Google account.
     - Current chart creation quota (e.g., "15/20 charts remaining").
     - Account creation date.
     - Last login timestamp.
  3. User reviews the displayed information.
- **Alternate Flows**:
  - **Profile Data Unavailable**: If the system cannot retrieve data (e.g., server error), it displays "Profile temporarily unavailable."
- **Post-conditions**:
  a) User views their profile details.
- **Activity Diagram**:



MyCharts: SRS

4. Non-Functional Requirements

4.1 Performance

- Chart generation completes within **3 seconds**.

4.2 Usability

- **Intuitive UI** with guided workflows (e.g., tooltips, validation hints) to assist non-technical users.

4.3 Security

- **SSL encryption**: Data transmitted ove the internet is encrypted using SSL.

4.4 Availability

- The service should be available 90% of the time on a daily basis.
- The system should prioritize **Availability over Consistency** during network partitioning.
- While in network partitioning, although temporary data inconsistencies may occur, as many as possible services should remain fully operational.

4.5 Scalability

- Support **1,000 concurrent users** with linear performance scaling via cloud-based auto-scaling infrastructure.

MyCharts: SRS

5. Data Requirements

5.1 Data Models

- **User Profile**: Store the following attributes for every user:
  - Name (from Google account)
  - Profile picture (from Google account)
  - Email (from Google account)
  - Remaining quota
  - Account creation timestamp
  - Last login timestamp
- **Chart Metadata**: Chart ID, type, creation date, Highcharts json chart object.

5.2 Data management

Data will be stored in databases (e.g., PostgreSQL, MySQL, MongoDB, etc), as required by the architecture to be implemented.

5.3 Data consistency

- After network partitioning, data should be synchronized to achieve a consistent state across the system's components.

MyCharts: SRS

6. Interface Requirements

6.1 User Interfaces

Web app, offering a frontend containing the following elements:

1. **Landing Page**

- Service description and features
- Sample chart gallery
- Pricing information
- Call-to-action buttons for login-registration

2. **Authentication Page**

- Google OAuth login button

3. **Profile Dashboard**

- User details from Google account
- Current quota status
- Account statistics (creation date, last login)
- Profile picture display

MyCharts: SRS

4. **Chart Management**

- View of previously generated charts
- Preview thumbnails
- Download/delete actions

5. **Chart Generator**

- Chart type selector
- CSV template downloader
- File upload zone with validation
- Real-time chart preview
- Format selection for export

6. **Quota Store**

- Quota selection
- Pricing display
- Payment gateway integration

6.2 External Interfaces

- Google OAuth for authentication.

MyCharts: SRS

7. System Constraints

7.1 Design Constraints

- Frontend: React Framework.
- Backend: Use of REST APIs, API gateways, Load balancer as needed.

7.2 Environmental Constraints

- Requires modern browser support.

8. Verification and Validation

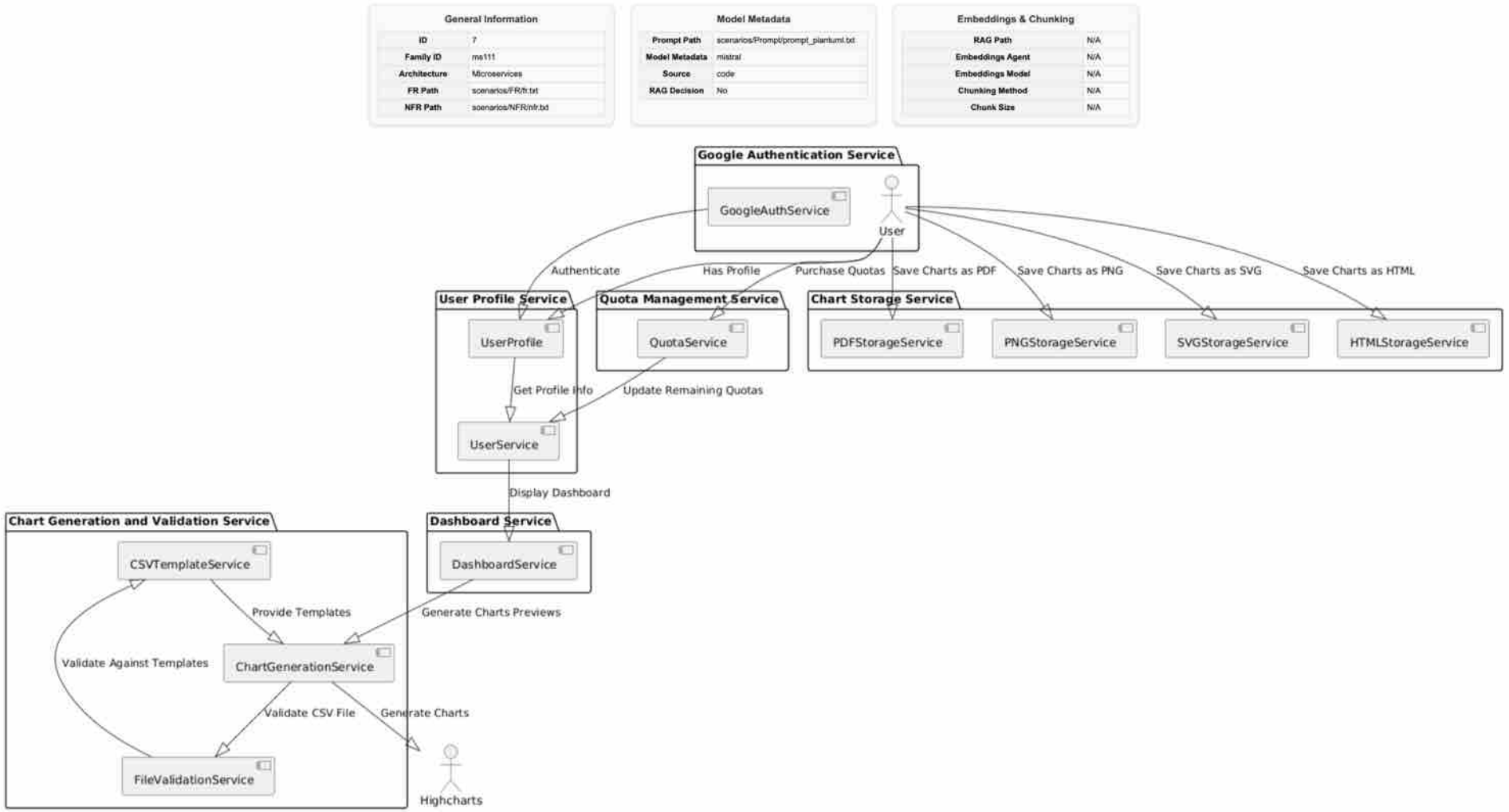
8.1 Verification Plan

- Unit tests for chart generation logic.
- Integration tests for Google OAuth.

8.2 Validation Plan

- User testing for ease of CSV upload and chart customization.

MyCharts: MS architectures by LLMs



MyCharts: MS architectures by LLMs

General Information

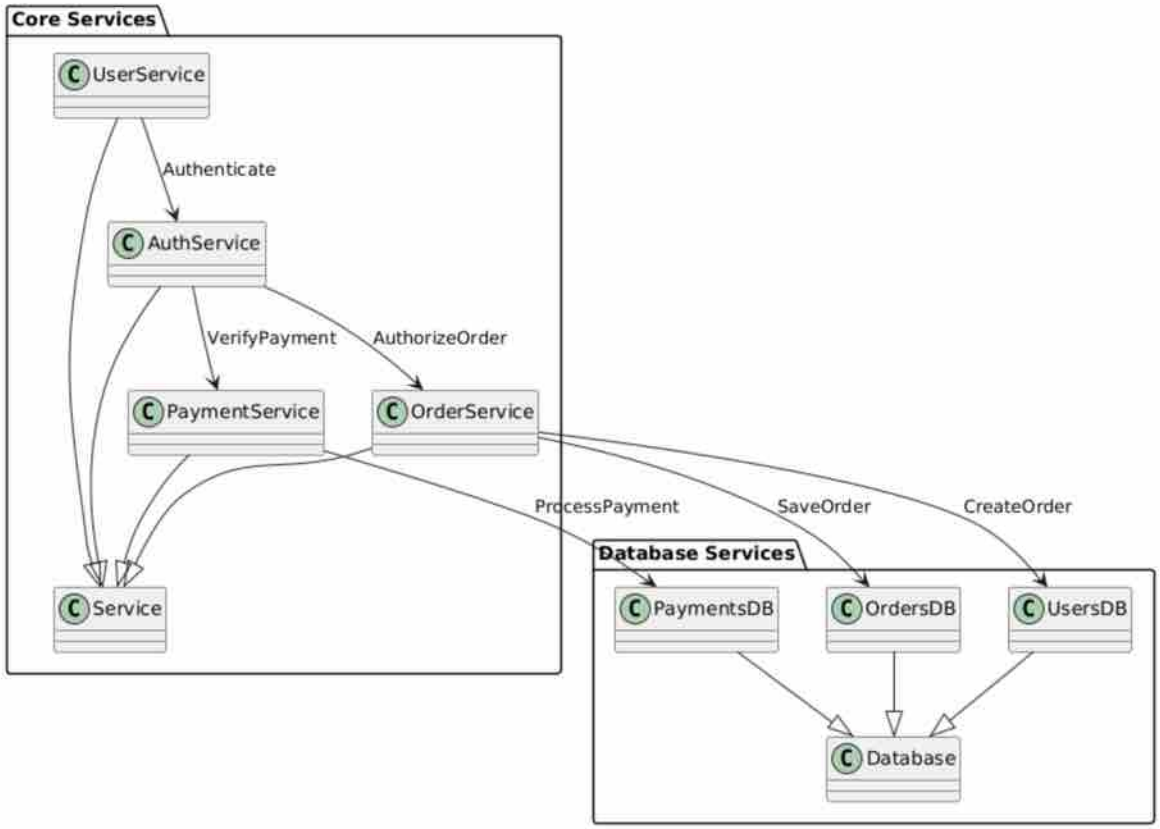
ID	8
Family ID	ms111
Architecture	Microservices
FR Path	scenarios/FR/fr.txt
NFR Path	scenarios/NFR/nfr.txt

Model Metadata

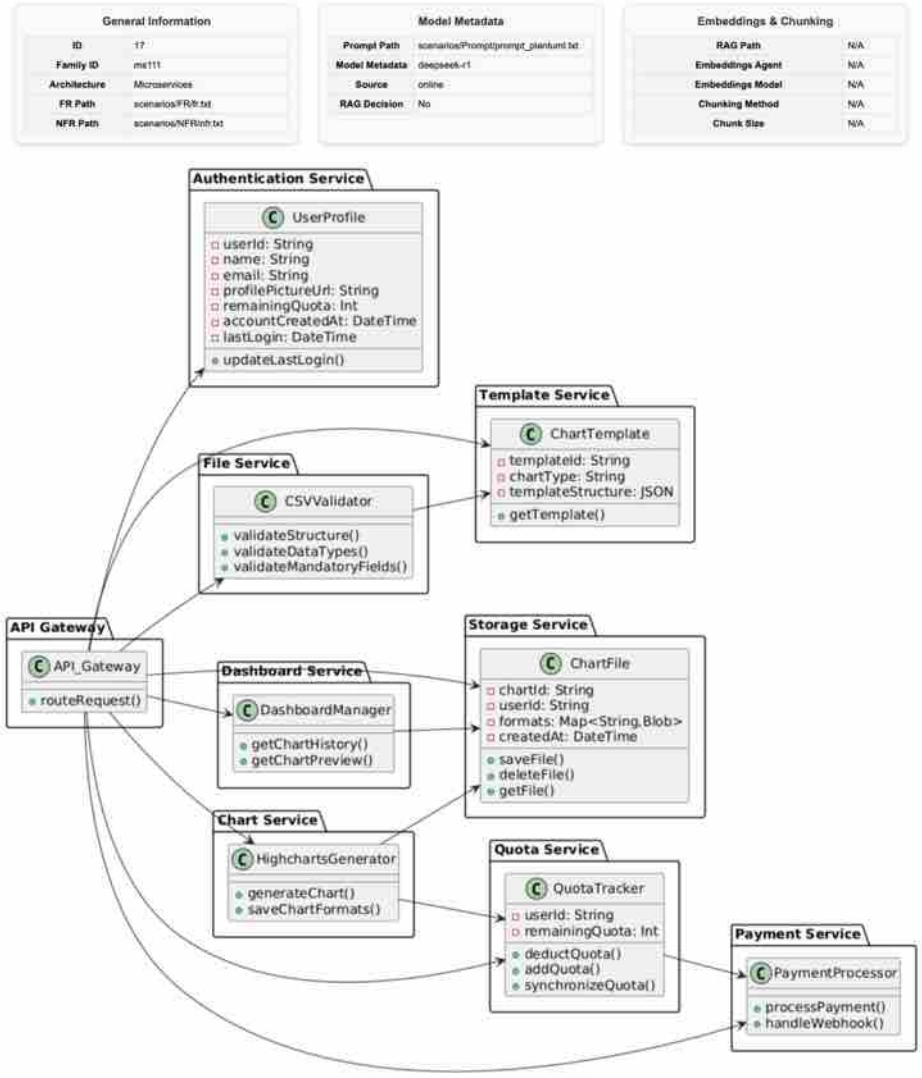
Prompt Path	scenarios/Prompt/prompt_plantuml.txt
Model Metadata	mistral
Source	code
RAG Decision	Yes

Embeddings & Chunking

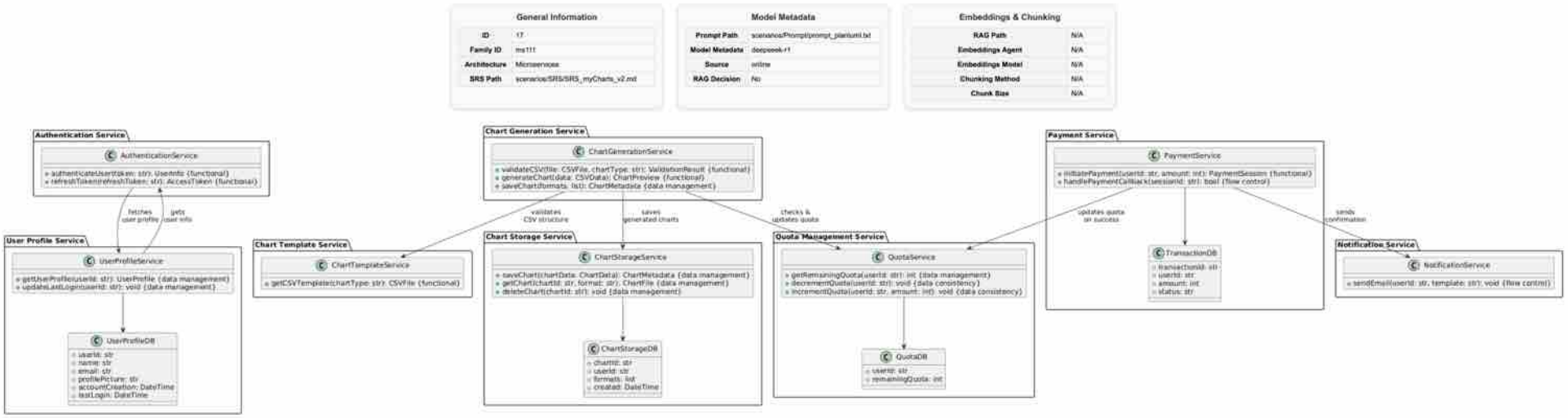
RAG Path	scenarios/RAG_Files/rag1.pdf
Embeddings Agent	ollama
Embeddings Model	nomic-embed-text:latest
Chunking Method	semantic
Chunk Size	-



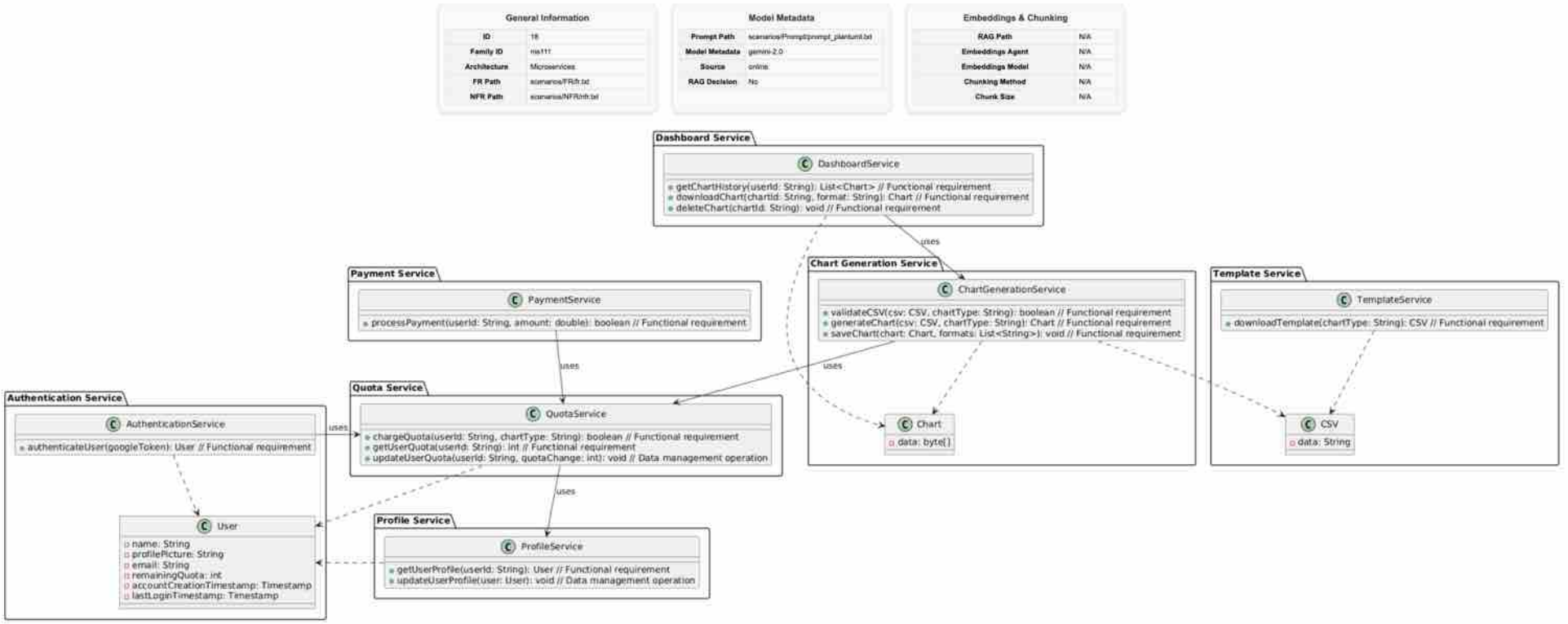
MyCharts: MS architectures by LLMs



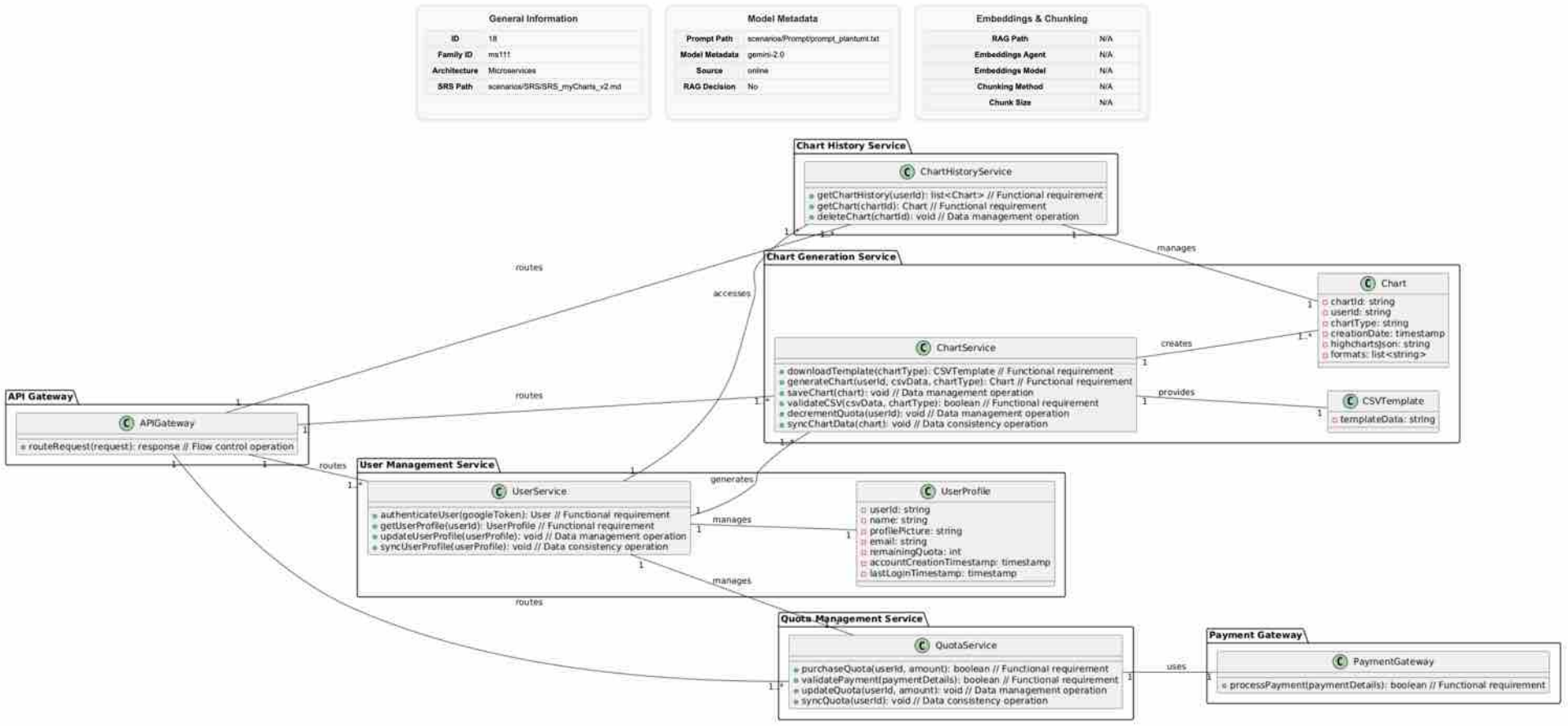
MyCharts: MS architectures by LLMs



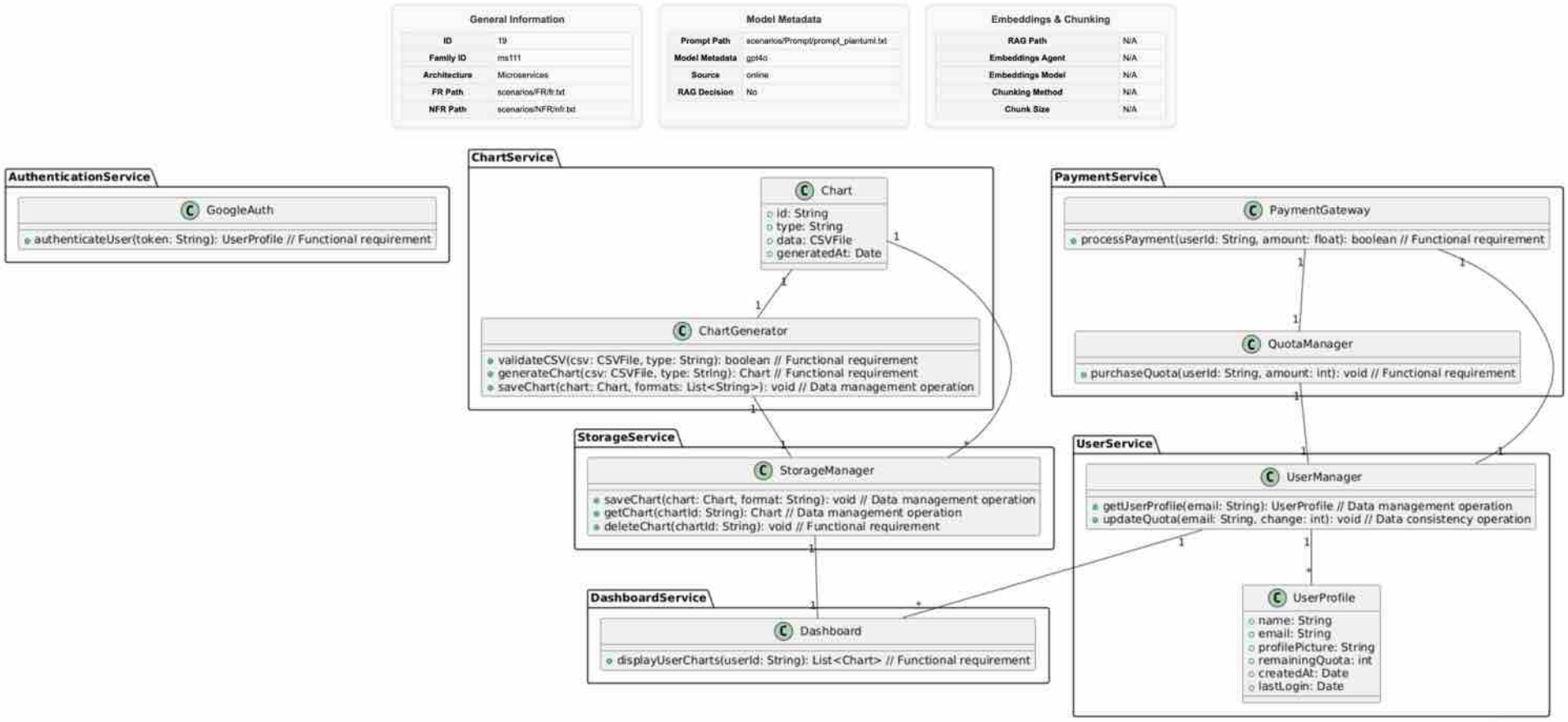
MyCharts: MS architectures by LLMs



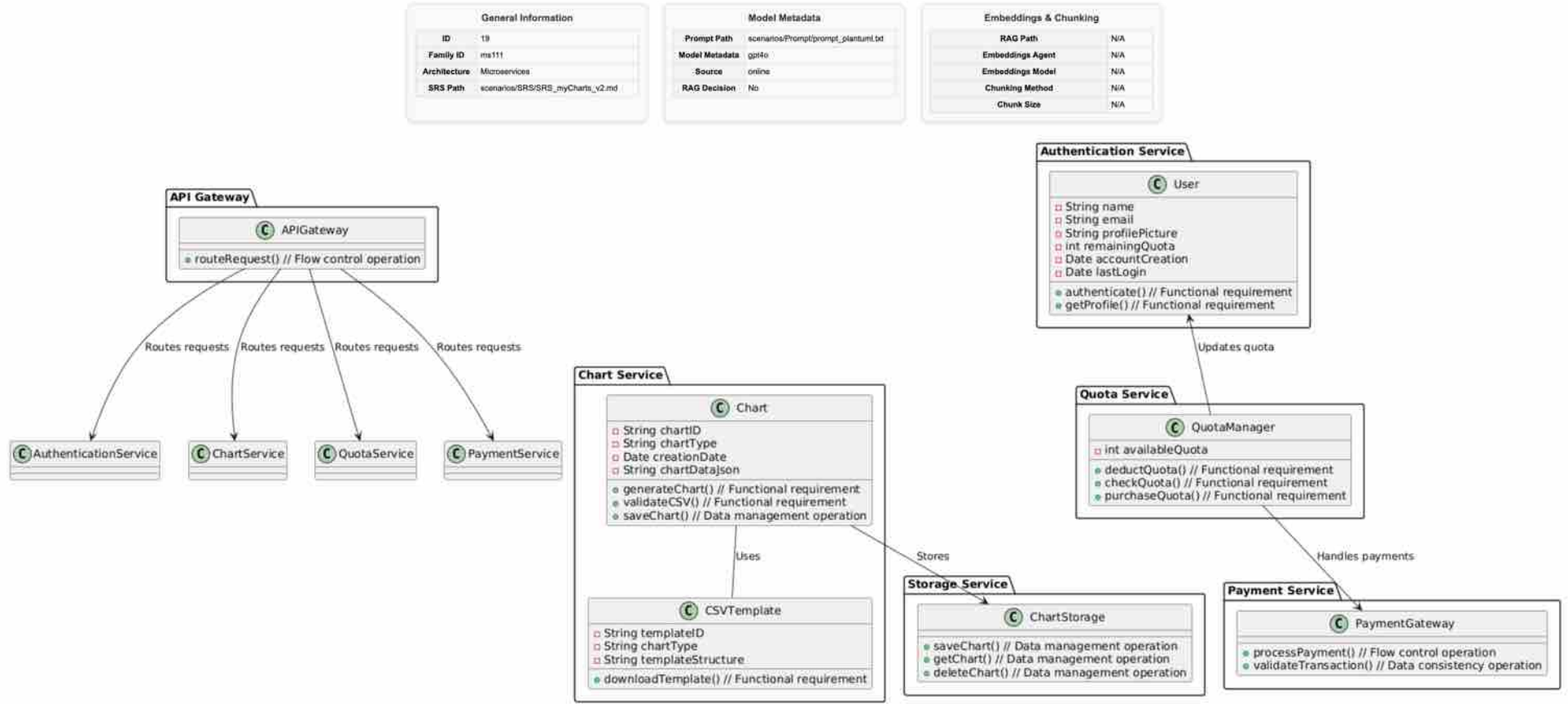
MyCharts: MS architectures by LLMs



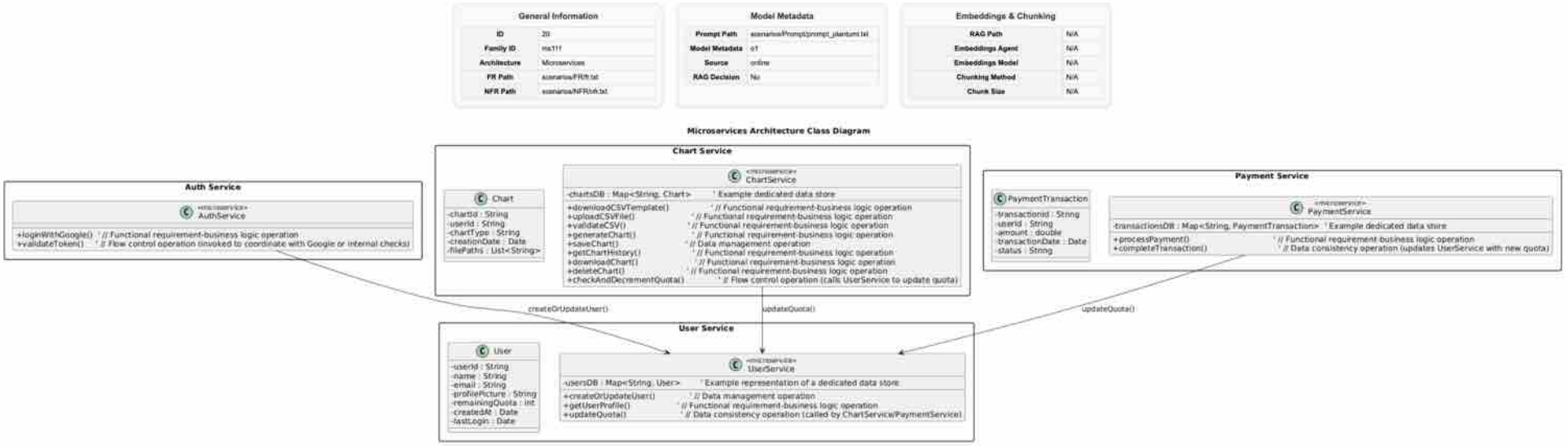
MyCharts: MS architectures by LLMs



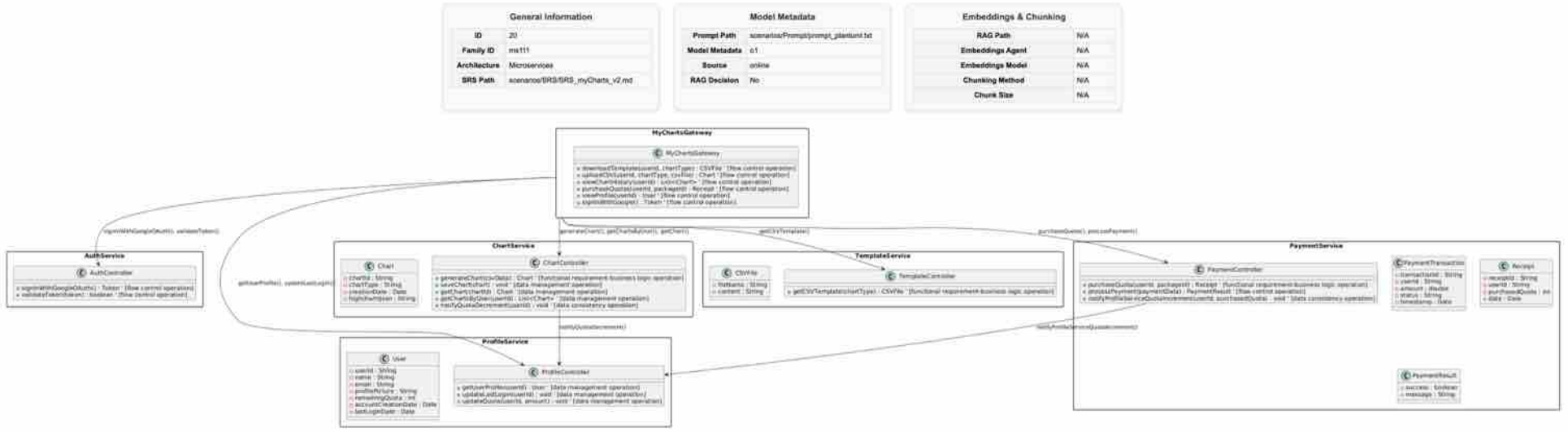
MyCharts: MS architectures by LLMs



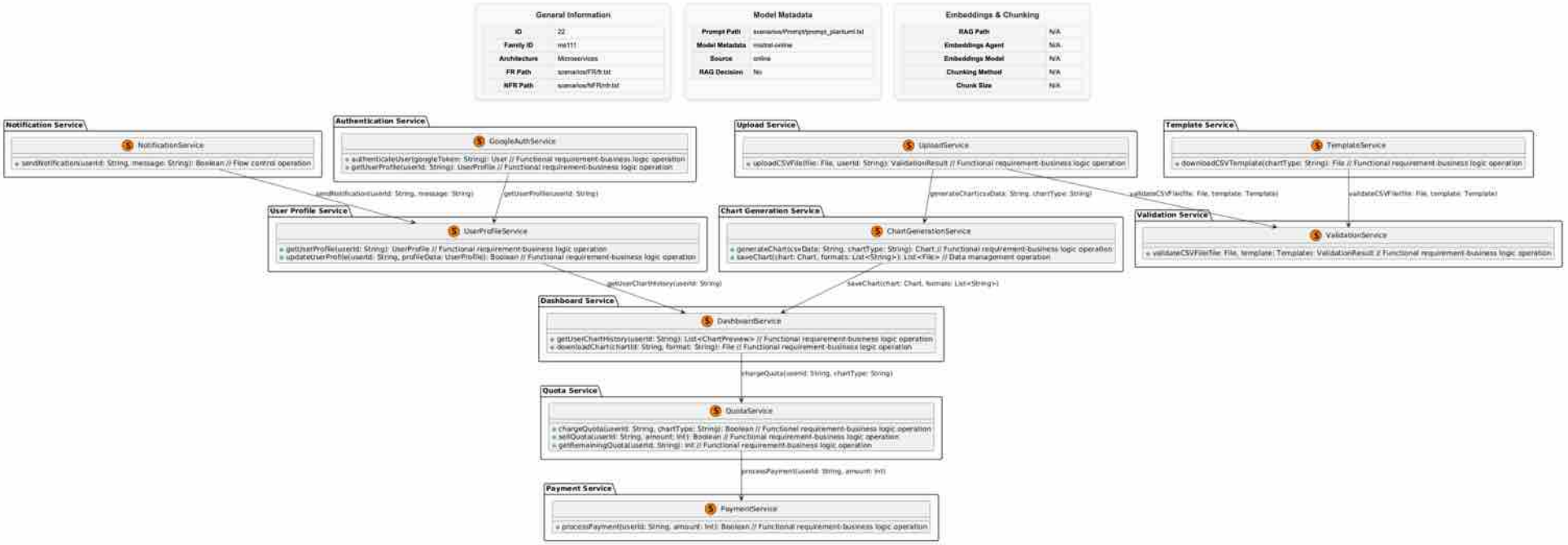
MyCharts: MS architectures by LLMs



MyCharts: MS architectures by LLMs



MyCharts: MS architectures by LLMs

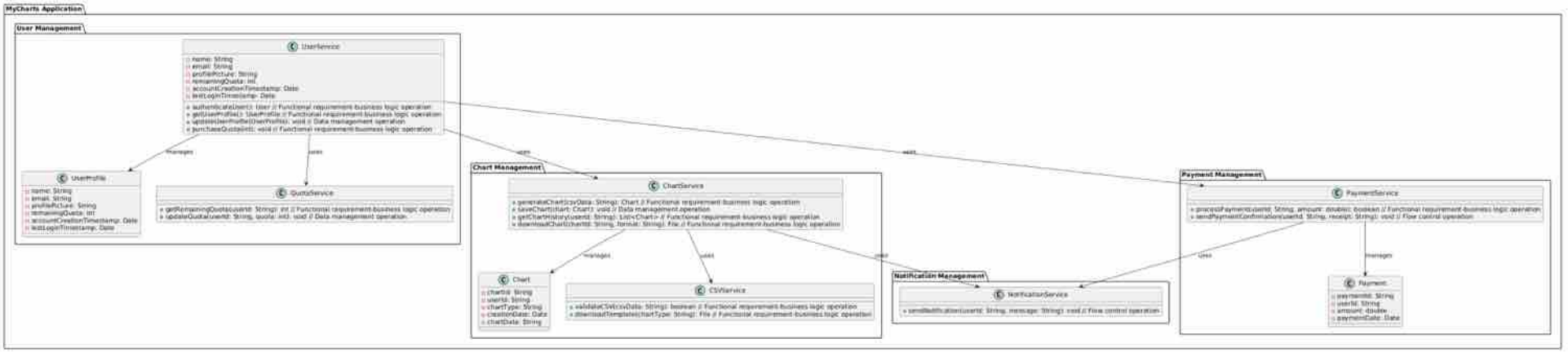


MyCharts: MS architectures by LLMs

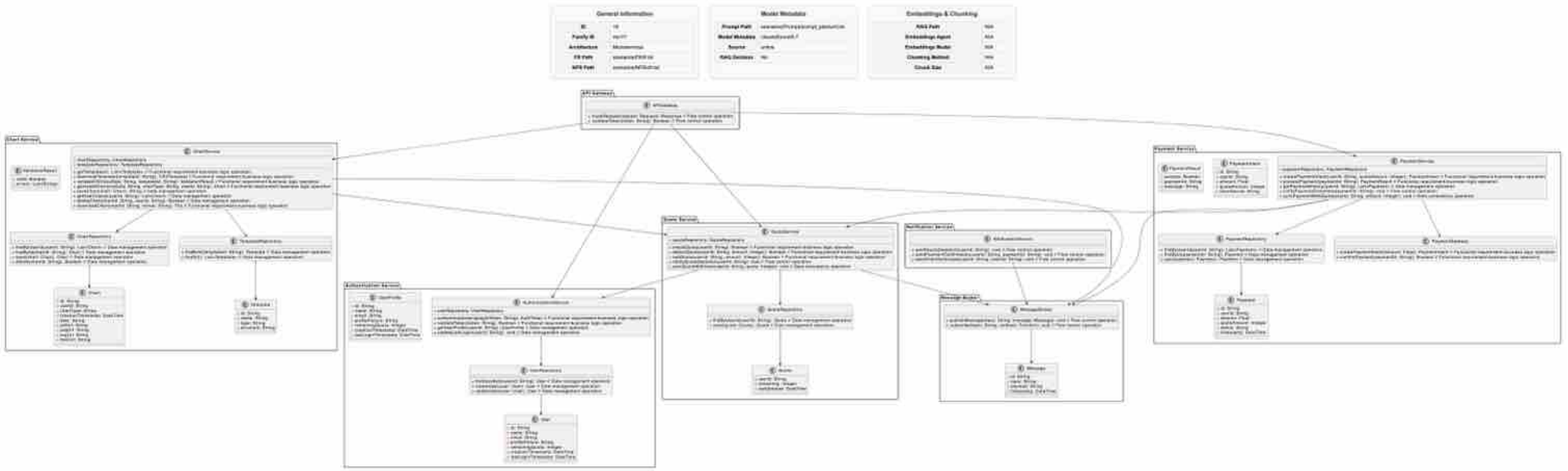
General Information	
ID	22
Family ID	ms111
Architecture	Microservices
SRS Path	scenarios/SRS/SRS_myCharts_v2.md

Model Metadata	
Prompt Path	scenarios/Prompt/prompt_plantuml.txt
Model Metadata	mistral-online
Source	online
RAG Decision	No

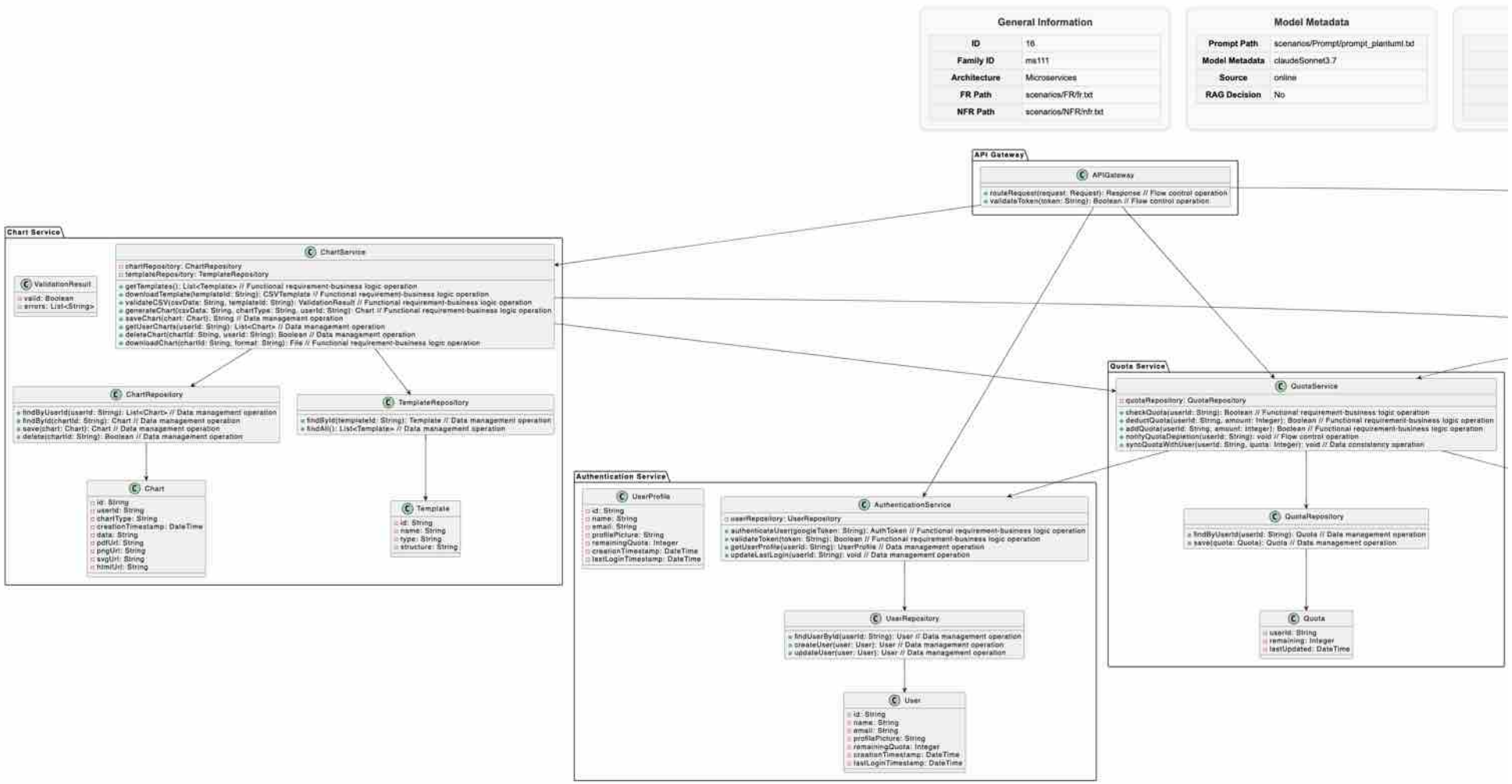
Embeddings & Chunking	
RAG Path	N/A
Embeddings Agent	N/A
Embeddings Model	N/A
Chunking Method	N/A
Chunk Size	N/A



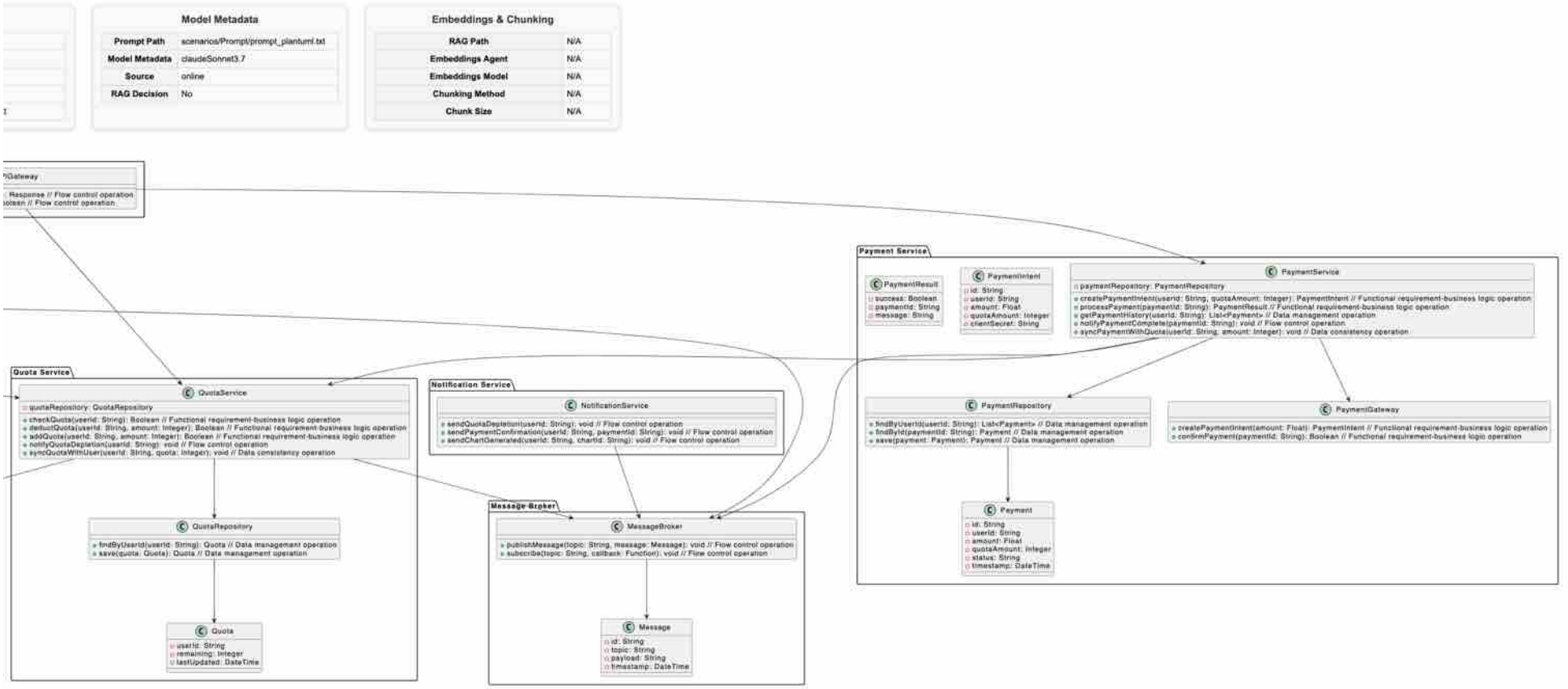
MyCharts: MS architectures by LLMs



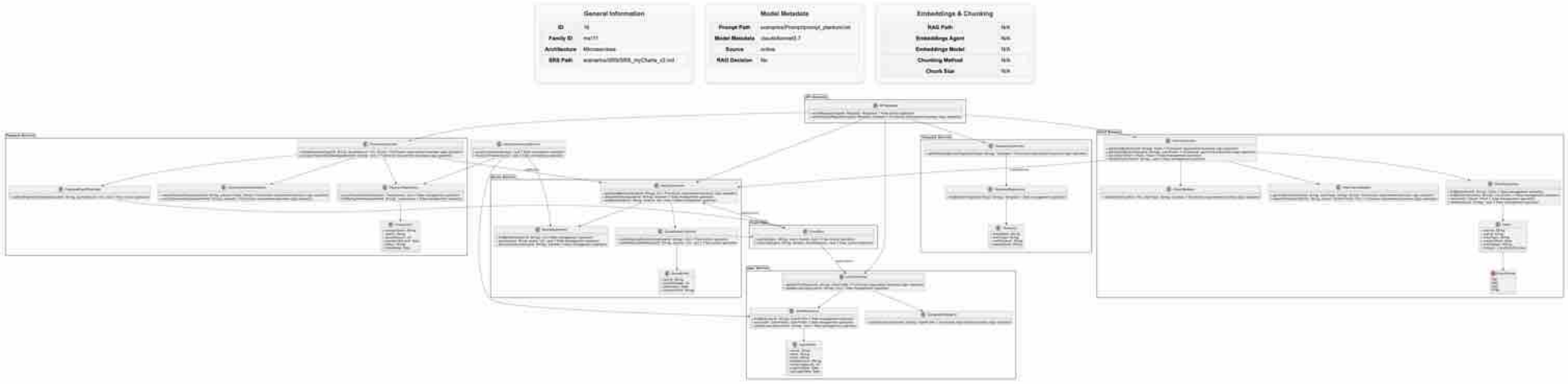
MyCharts: MS architectures by LLMs



MyCharts: MS architectures by LLMs



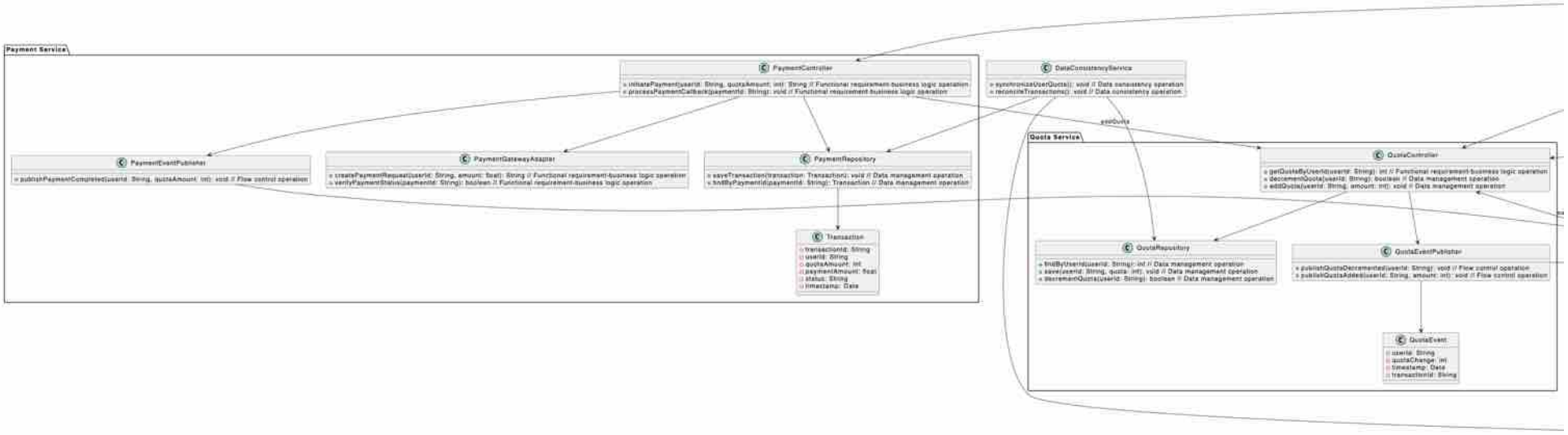
MyCharts: MS architectures by LLMs



MyCharts: MS architectures by LLMs

General Information

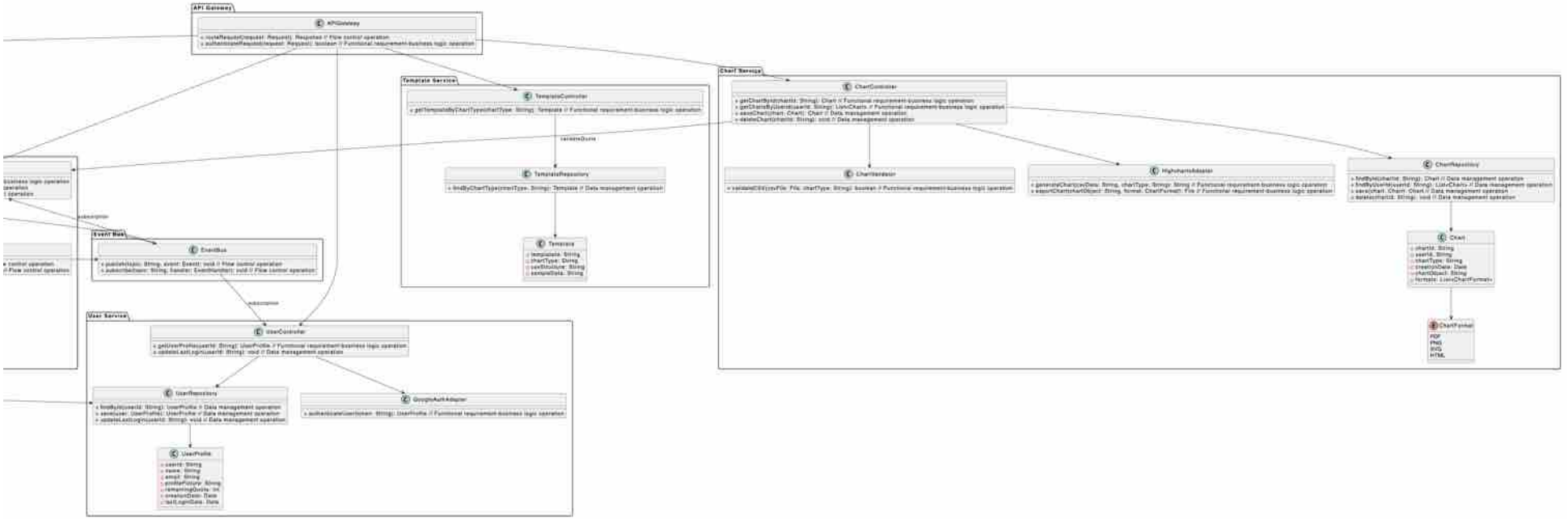
ID	16
Family ID	ms111
Architecture	Microservices
SRS Path	scenarios/SRS/SRS_myCharts_v2.md



MyCharts: MS architectures by LLMs

Model Metadata	
Prompt Path	scenarios/Prompt/prompt_plantuml.txt
Model Metadata	claudeSonnet3.7
Source	online
RAG Decision	No

Embeddings & Chunking	
RAG Path	N/A
Embeddings Agent	N/A
Embeddings Model	N/A
Chunking Method	N/A
Chunk Size	N/A



You have access, too!

https://saai.softlab.ntua.gr/experiments/mycharts

